

Optimization Algorithms



CS4234

Week 2

Set Cover, Vertex Cover,
Traveling Salesperson

Diptarka Chakraborty

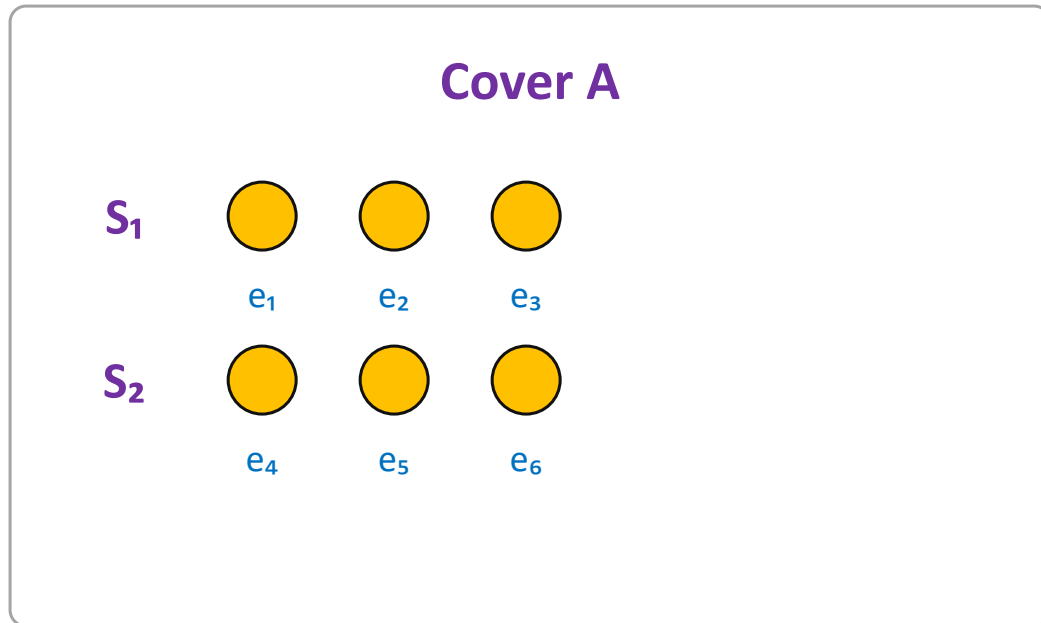
Set cover: choose sets that cover every element

Universe $U = \{e_1, e_2, e_3, e_4, e_5, e_6\}$. Each row below is one available set.

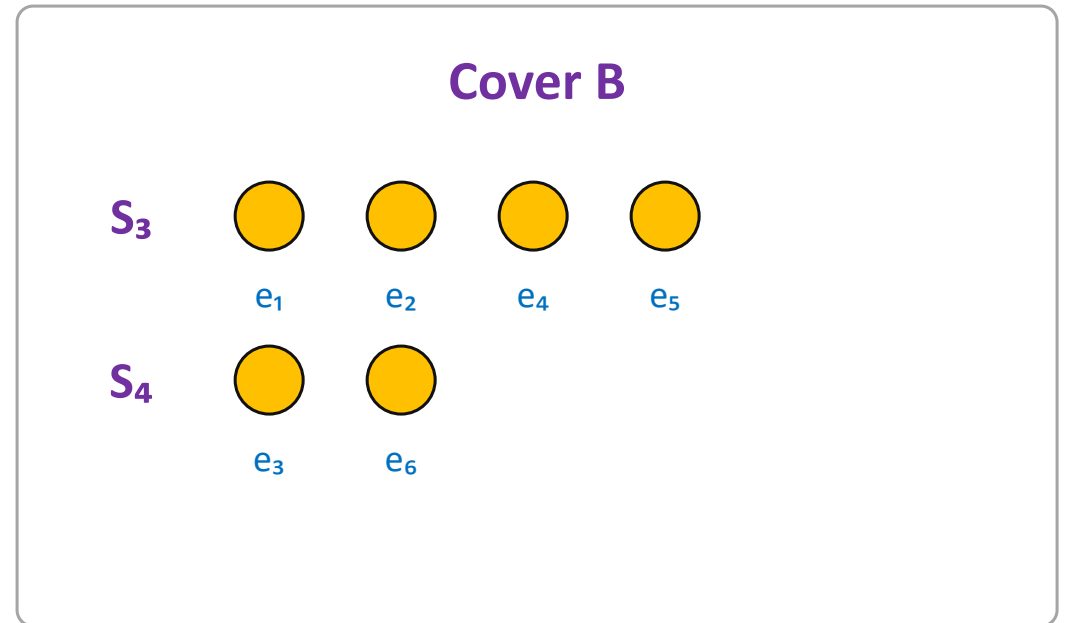
Set	e_1	e_2	e_3	e_4	e_5	e_6
S_1						
S_2						
S_3						
S_4						

Goal: choose the fewest rows whose dots/elements touch every column.

Two different covers use the same number of sets



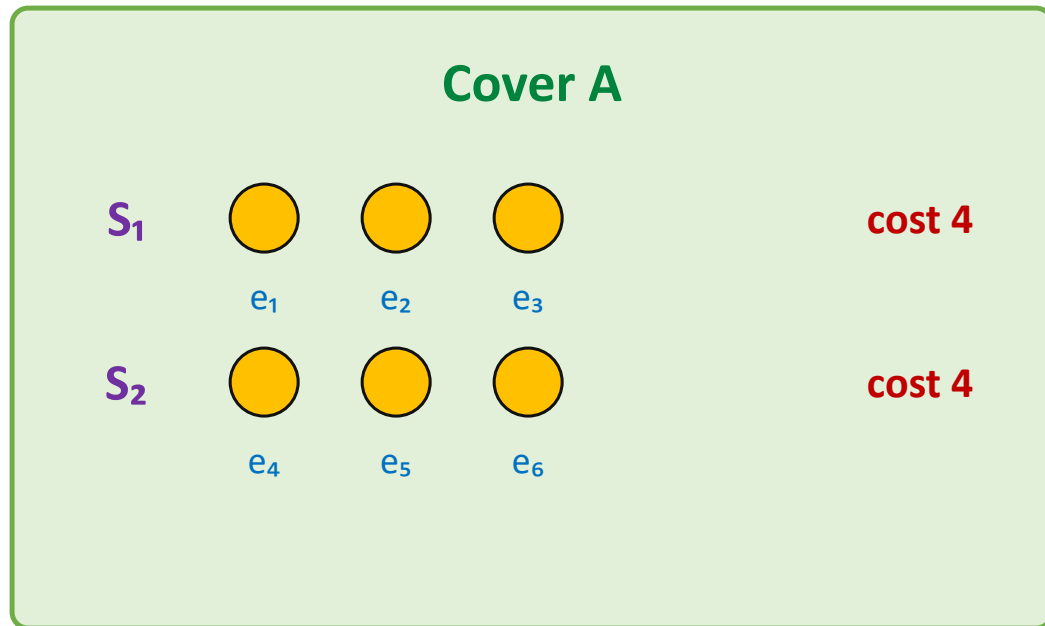
2 sets



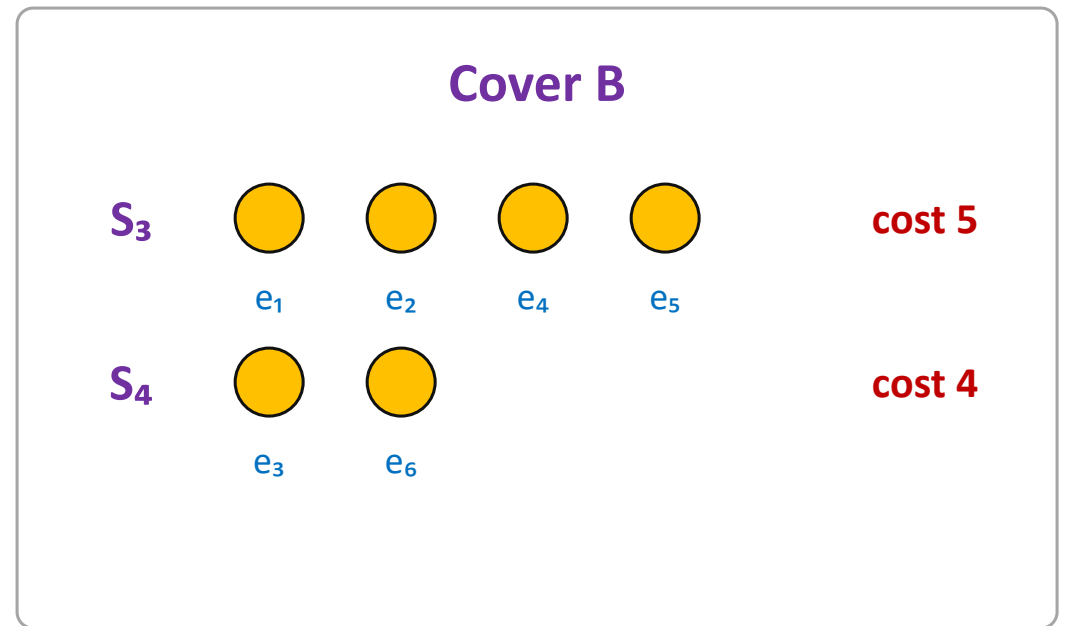
2 sets

In unweighted set cover, both solutions are equally good.

Costs distinguish equally small covers



total cost = 8



total cost = 9

Weighted set cover minimizes total cost—not the number of sets.

Exact optimization is hard—seek a guarantee

Decision version

Given a budget B , is there a
cover
with total cost at most B ?

NP-complete



Approximation

Find a polynomial-time solution
and prove its worst-case ratio.

$\text{cost}(\text{ALG}) \leq \rho \cdot \text{OPT}$

We will prove a logarithmic guarantee for a simple greedy rule.

Greedy buys the cheapest newly covered element

R = elements not yet covered



$$\text{score}(S) = \text{cost}(S) / |S \cap R|$$

- 1 Compute every score
- 2 **Choose a minimum-score set**
- 3 Remove its newly covered elements from R

Repeat until $R = \emptyset$.

Iteration 1: S_3 has the best current price

All six elements are initially uncovered.

Set	e_1	e_2	e_3	e_4	e_5	e_6	cost	cost / new
S_1							4	$4/3 = 1.33$
S_2							4	$4/3 = 1.33$
S_3							5	$5/4 = 1.25$
S_4							4	$4/2 = 2.00$

Pick S_3 ; it covers e_1, e_2, e_4, e_5 .

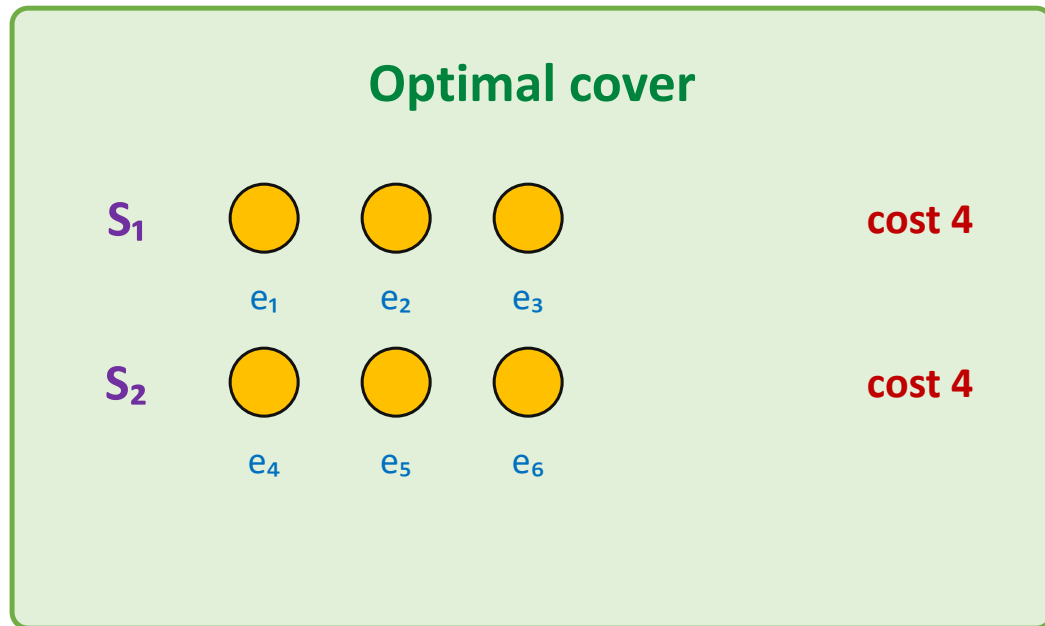
Iteration 2: only e_3 and e_6 remain

Grey columns are already covered by S_3 .

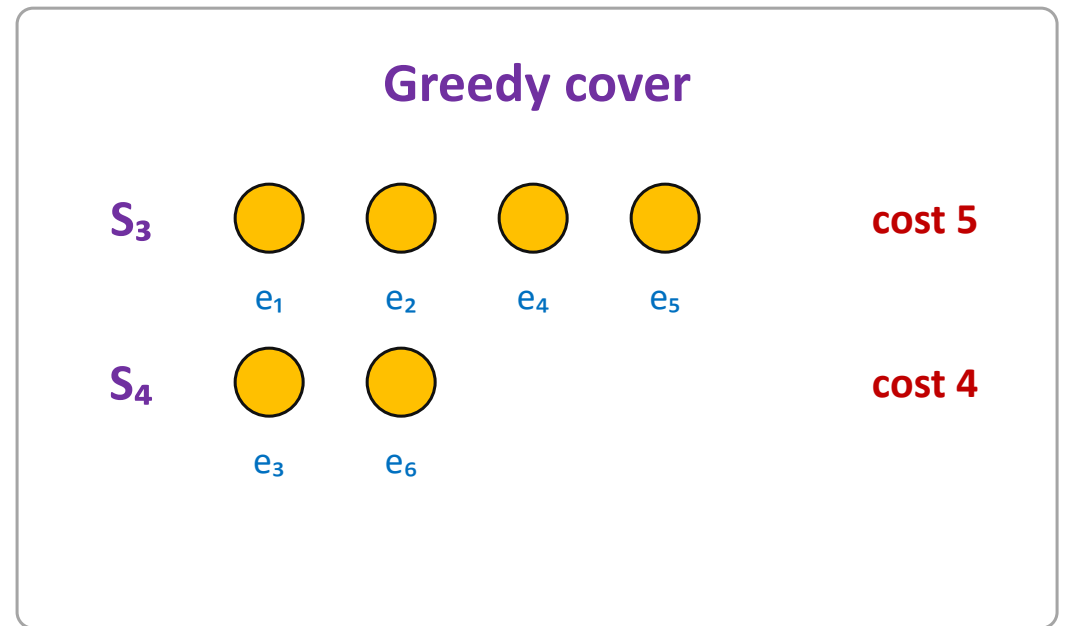
Set	e_1	e_2	e_3	e_4	e_5	e_6	cost	cost / new
S_1							4	$4/1 = 4.00$
S_2							4	$4/1 = 4.00$
S_3							5	—
S_4							4	$4/2 = 2.00$

Pick S_4 ; every element is now covered.

Greedy is close—but not optimal



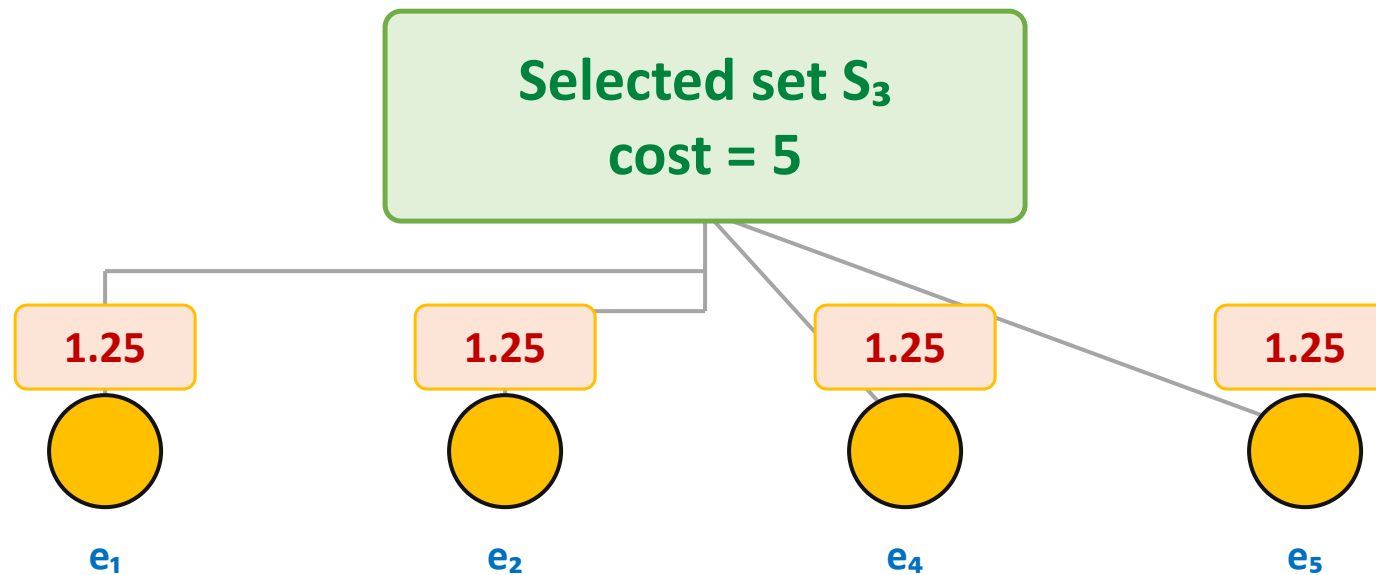
OPT = 8



GREEDY = 9

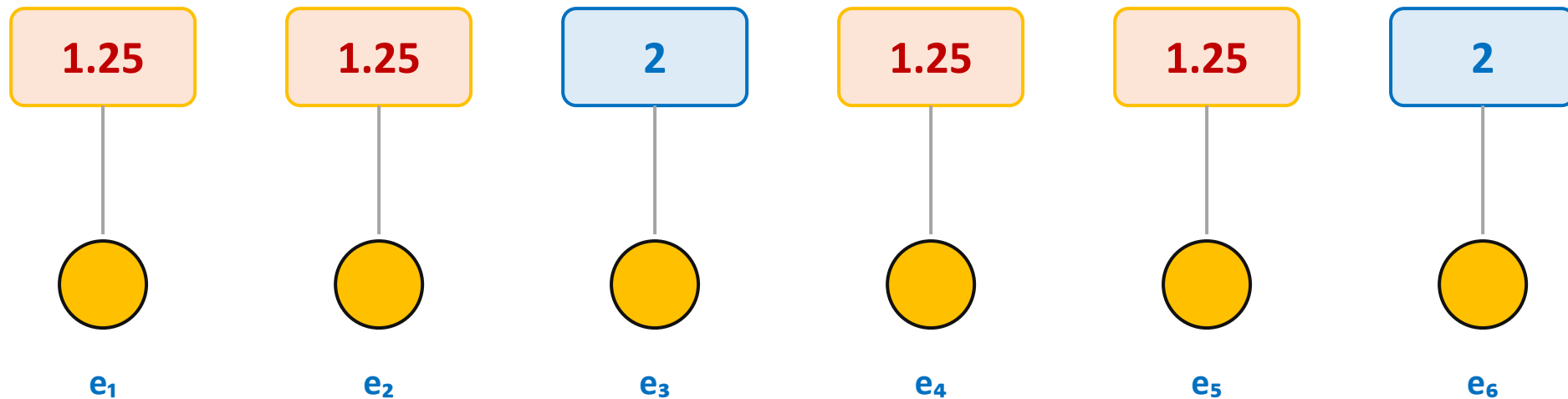
How can we bound GREEDY / OPT without knowing OPT?

Charge S_3 's cost equally to its new elements



$5 = 4 \times 1.25$ — each newly covered element receives price $p(e) = 1.25$

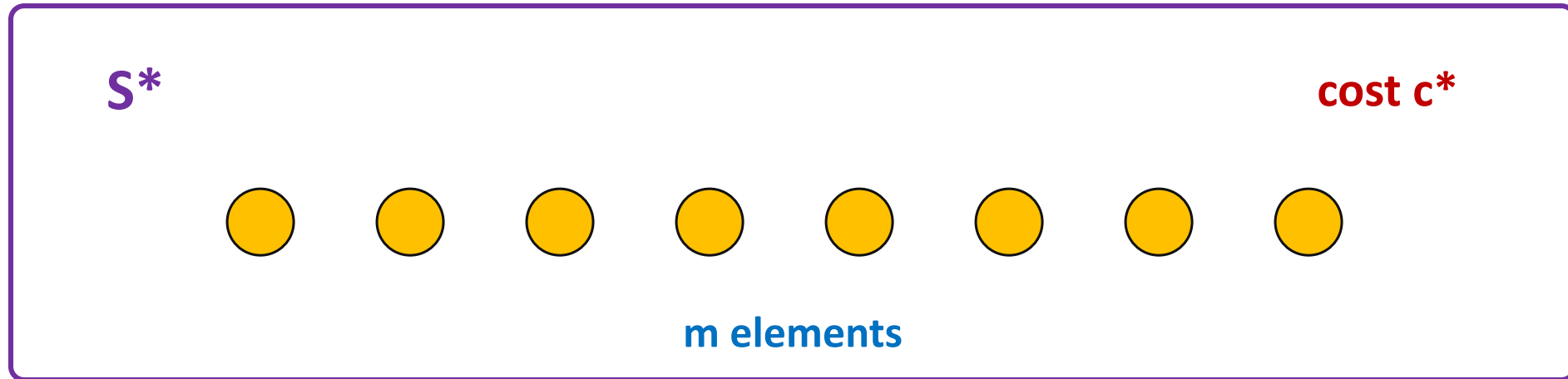
The element charges add up to greedy's cost



$$1.25 + 1.25 + 2 + 1.25 + 1.25 + 2 = 9$$

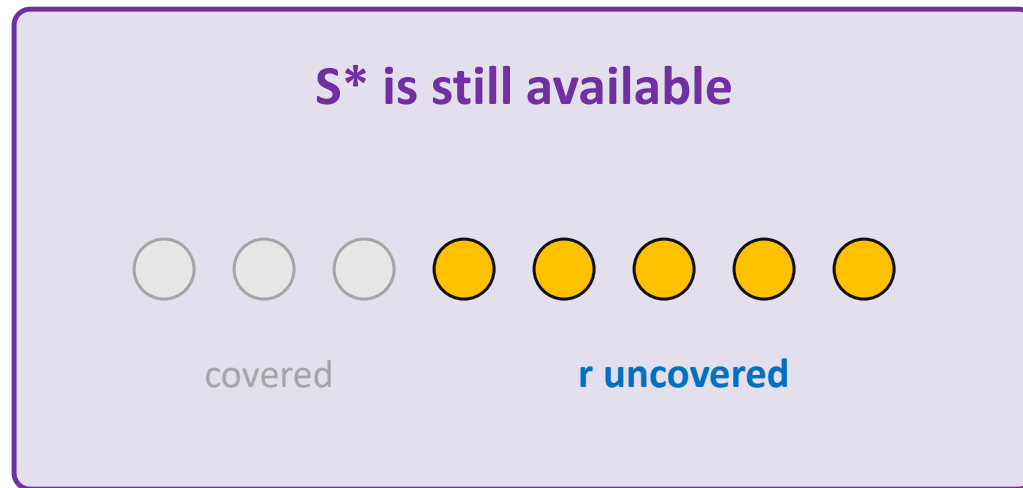
$$\sum_e p(e) = \text{cost}(\text{GREEDY})$$

Focus on one set S^* chosen by OPT



We will bound the total greedy charge paid by the elements of S^* .

With r elements left, greedy's price is at most c^*/r



S^* could cover the remaining r elements
at average cost c^*/r .

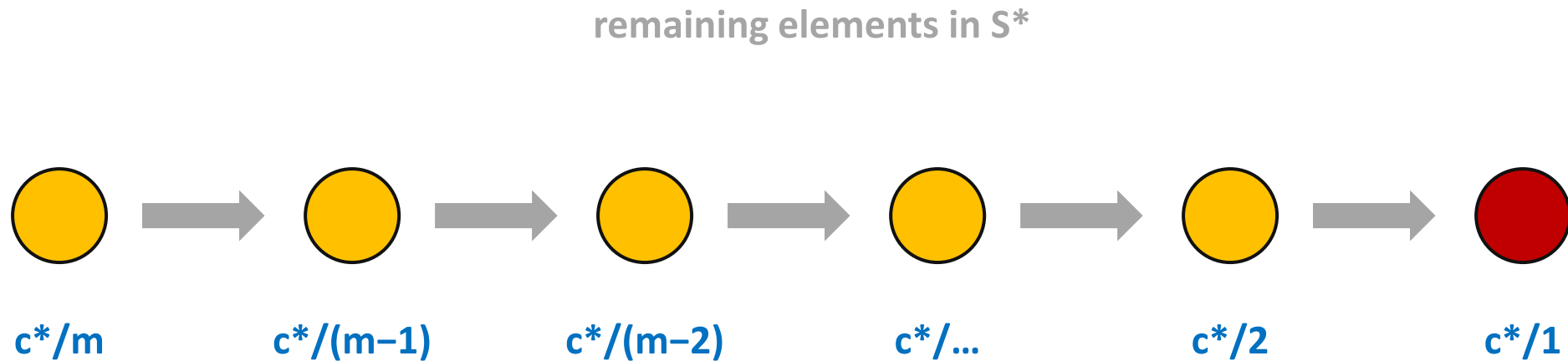


Greedy chooses a minimum score

therefore $p(\text{next element}) \leq c^*/r$

This inequality is the core of the analysis.

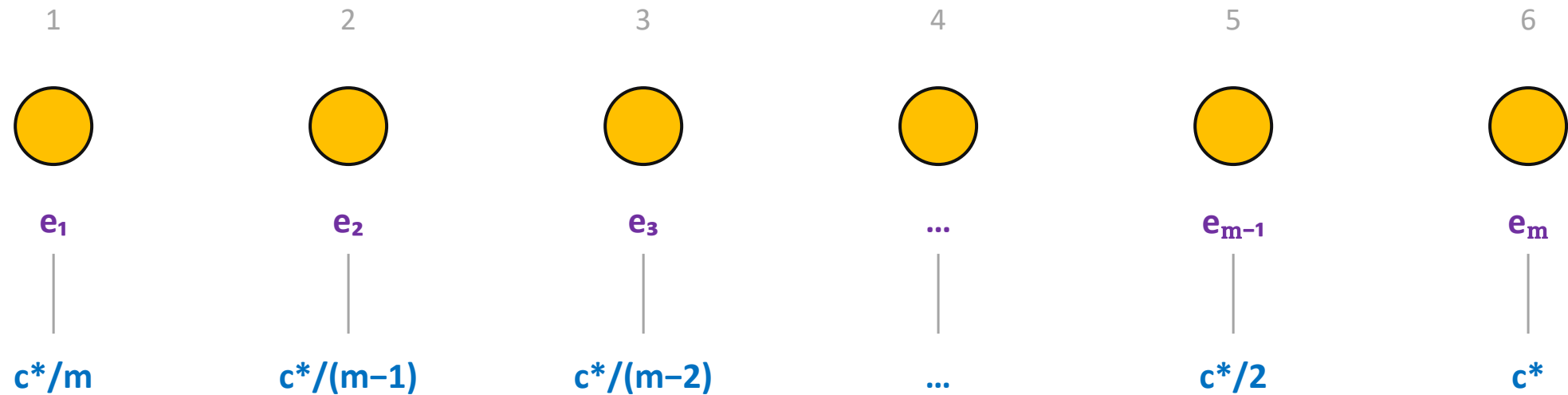
The harmonic series appears as elements are covered



Each denominator shrinks, so later elements may receive a larger charge.

Add these bounds—not the actual unknown optimal solution.

Order S^* 's elements by when greedy covers them



$p(e_t)$ is no larger than the cost of S^* divided by the number still uncovered.

$$p(e_t) \leq c^* / (m - t + 1)$$

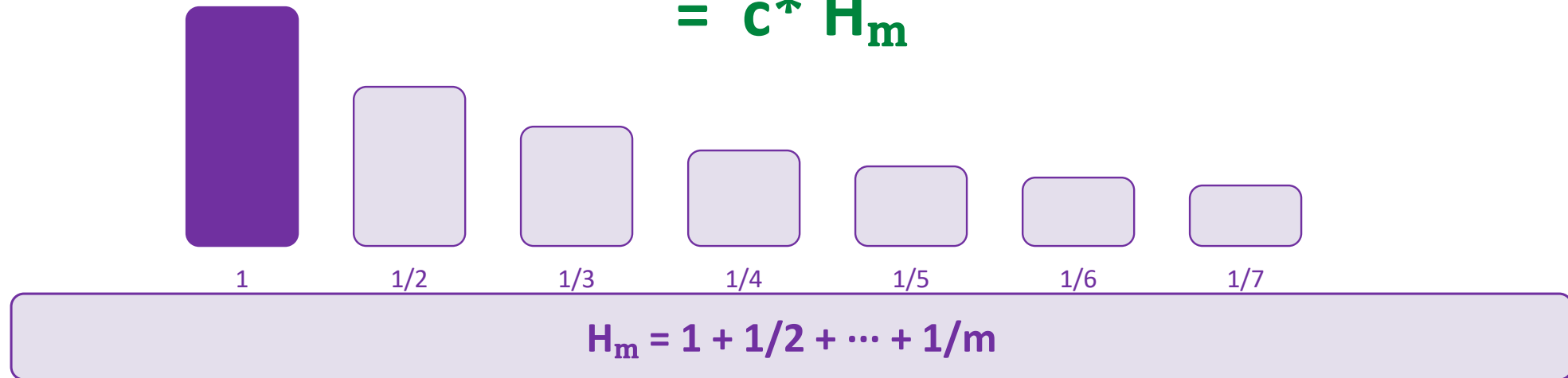
One optimal set is charged at most $c^* H_m$

$$\sum_{e \in S^*} p(e)$$

$\leq$

$$c^* \left(\frac{1}{m} + \frac{1}{m-1} + \dots + 1 \right)$$

$$= c^* H_m$$



Summing over OPT gives the approximation ratio

optimal set S_1^*
charge $\leq H_n \cdot \text{cost}(S_1^*)$

optimal set S_2^*
charge $\leq H_n \cdot \text{cost}(S_2^*)$

optimal set S_3^*
charge $\leq H_n \cdot \text{cost}(S_3^*)$



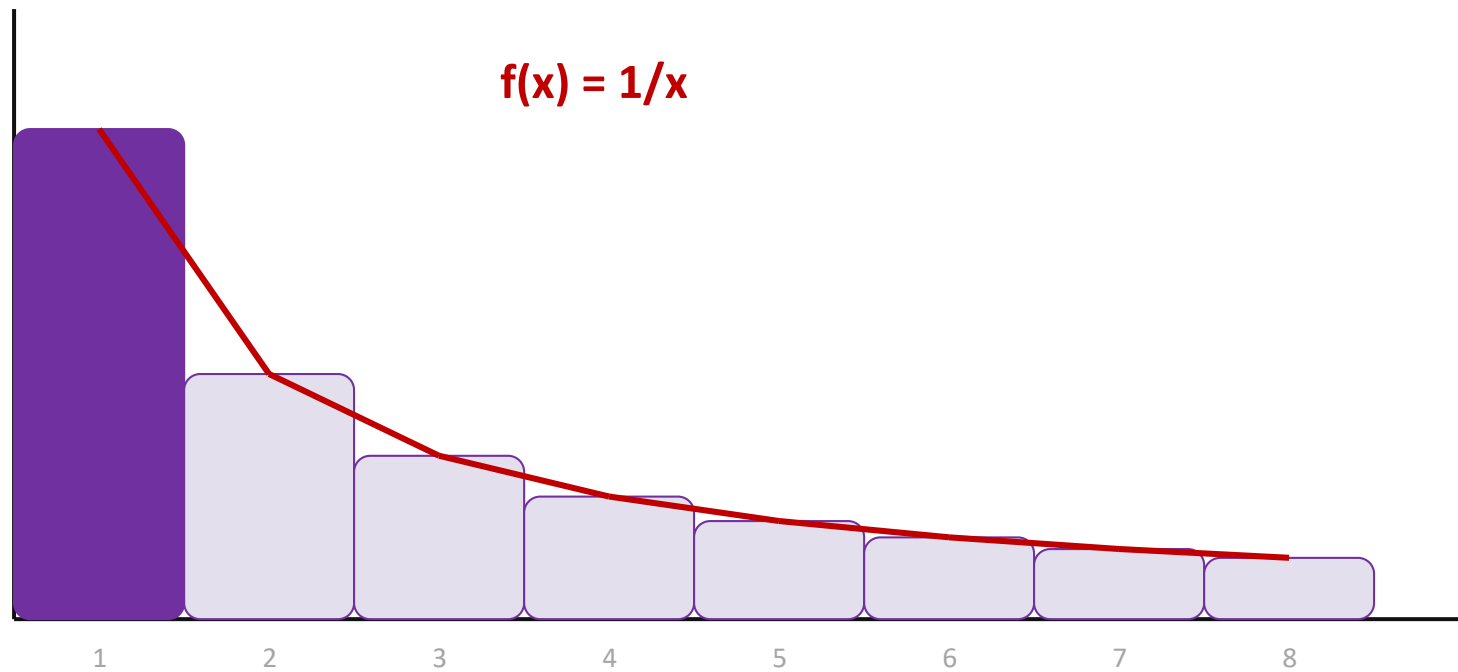
$$\text{GREEDY} = \sum_e p(e)$$

$$\leq H_n \cdot \sum_{S^* \in \text{OPT}} \text{cost}(S^*)$$

$$= H_n \cdot \text{OPT}$$

Every element is covered by at least one optimal set.

The harmonic number grows logarithmically



$$H_n = 1 + 1/2 + \dots + 1/n$$

$$\leq 1 + \int_1^n dx/x$$

$$= 1 + \ln n$$

$$\text{GREEDY} \leq H_n \cdot \text{OPT} \leq (1 + \ln n) \cdot \text{OPT}$$

Greedy set cover is logarithmically optimal

Algorithm

Choose the set with minimum cost per newly covered element.

Guarantee

$\text{cost}(\text{GREEDY}) \leq H_n \cdot \text{OPT} \leq (1 + \ln n) \cdot \text{OPT}.$

Limit

For every $\alpha > 0$, $(1 - \alpha) \ln n$ approximation is NP-hard.
[[Dinur & Steurer'14](#)]

A simple local choice achieves an essentially best-possible global guarantee.

Announcement

Tutorials

Tutorial sessions start next week.

Two groups

Thu 14:00–15:00 at LT15 | Tue 19:00–20:00 at **COM4 Seminar Room 31**

Problem set

Uploaded by Friday in the week before the tutorial.

For next week's tutorial, look for the problem set tomorrow.

Graph structure improves the guarantee

Generic set cover

Edges are elements.
Each vertex covers its incident
edges.

logarithmic



Use a matching

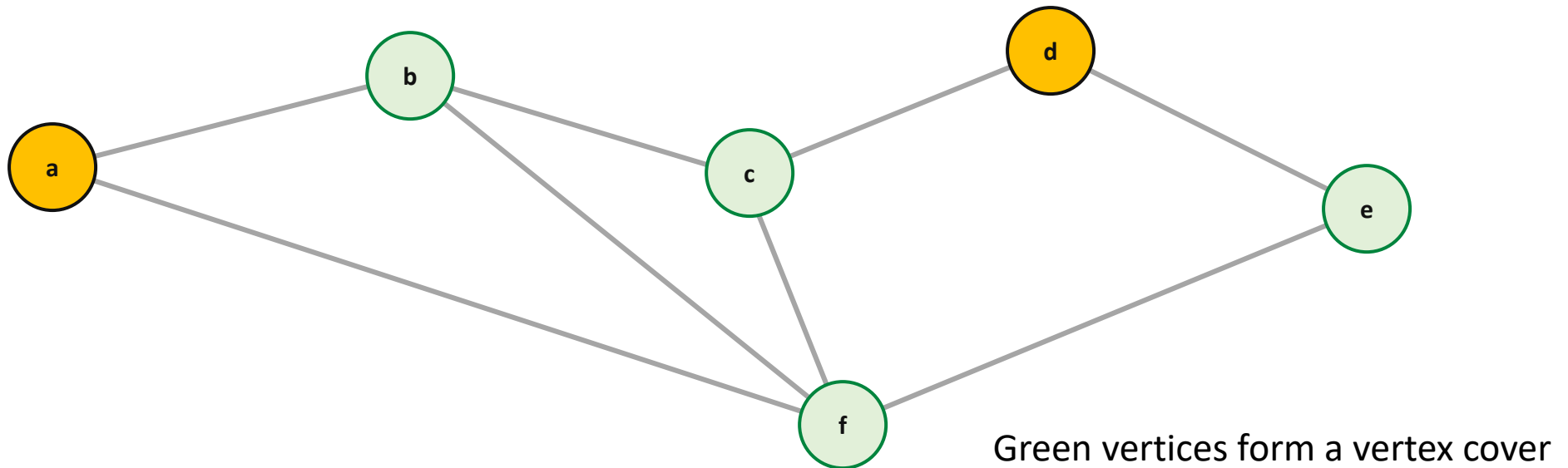
Disjoint edges certify a lower bound
on every cover.

factor 2

Same covering idea—stronger structure—better guarantee.

Vertex Cover: select vertices that touch every edge

A set $C \subseteq V$ is a vertex cover if every edge has at least one endpoint in C .



Goal: minimize $|C|$.

Maximal matching gives a 2-approximation

$M = \emptyset$



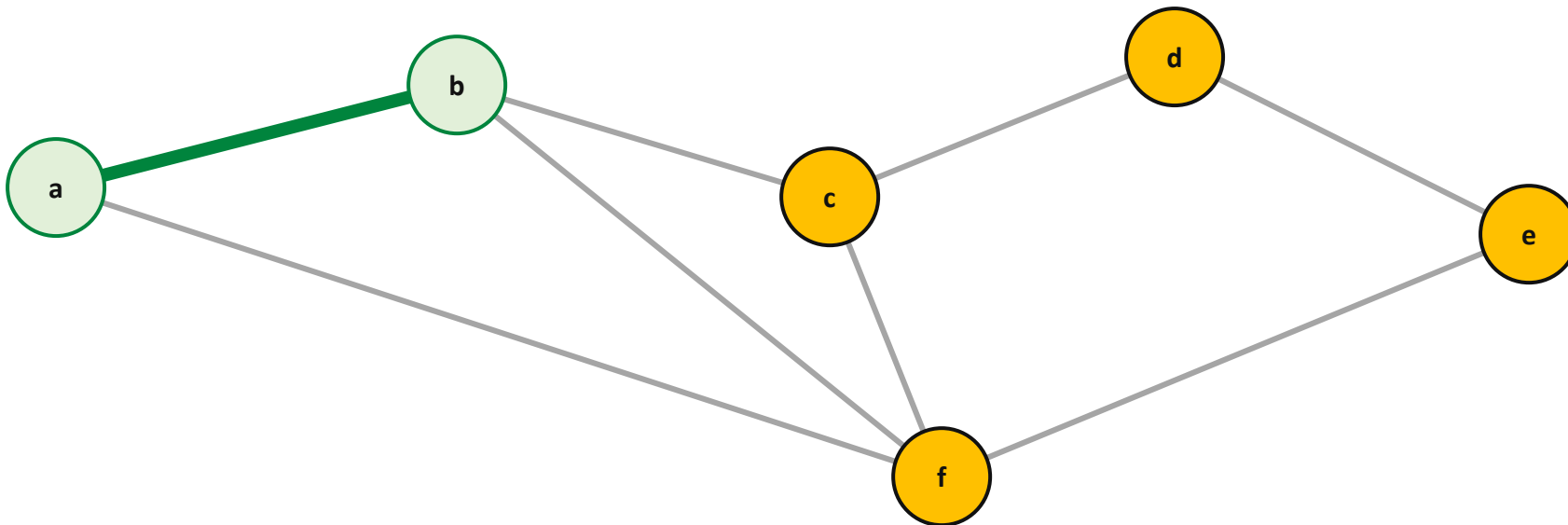
Return $C = \text{endpoints}(M)$

- 1 Choose any remaining edge e
- 2 **Add e to M**
- 3 Delete every edge incident to e

Repeat until no edge remains.

Build 1: choose any edge

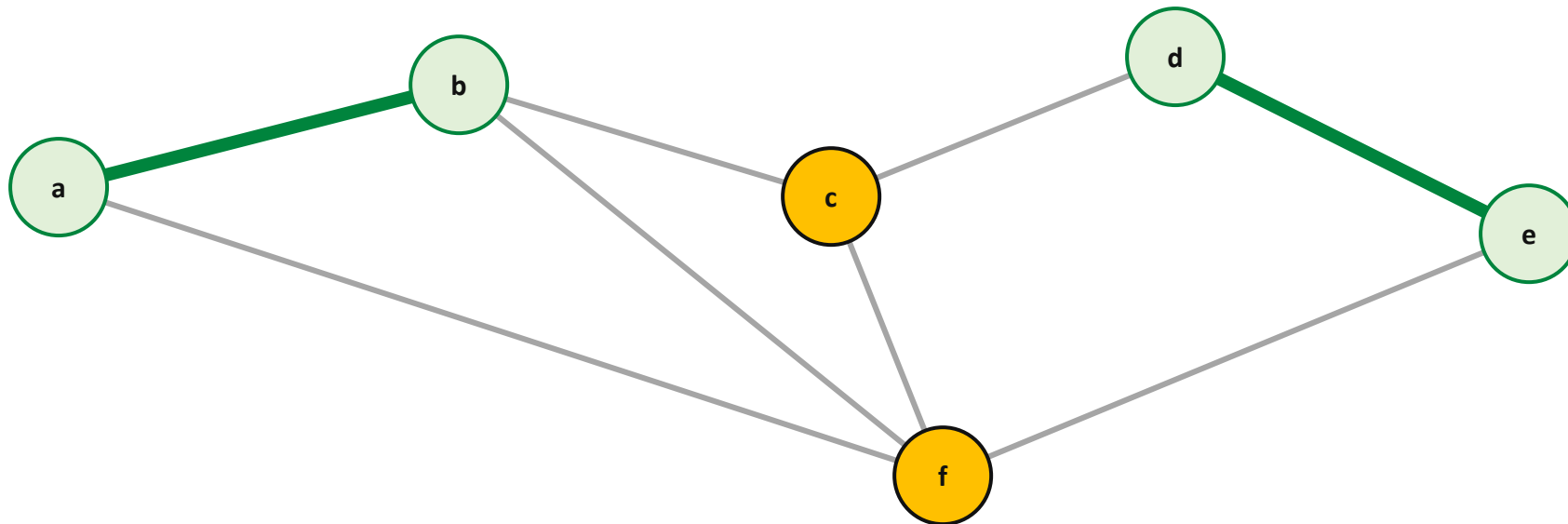
Start with $M = \emptyset$ and choose edge $\{a,b\}$.



Matched edges never share an endpoint.

Build 2: choose another disjoint edge

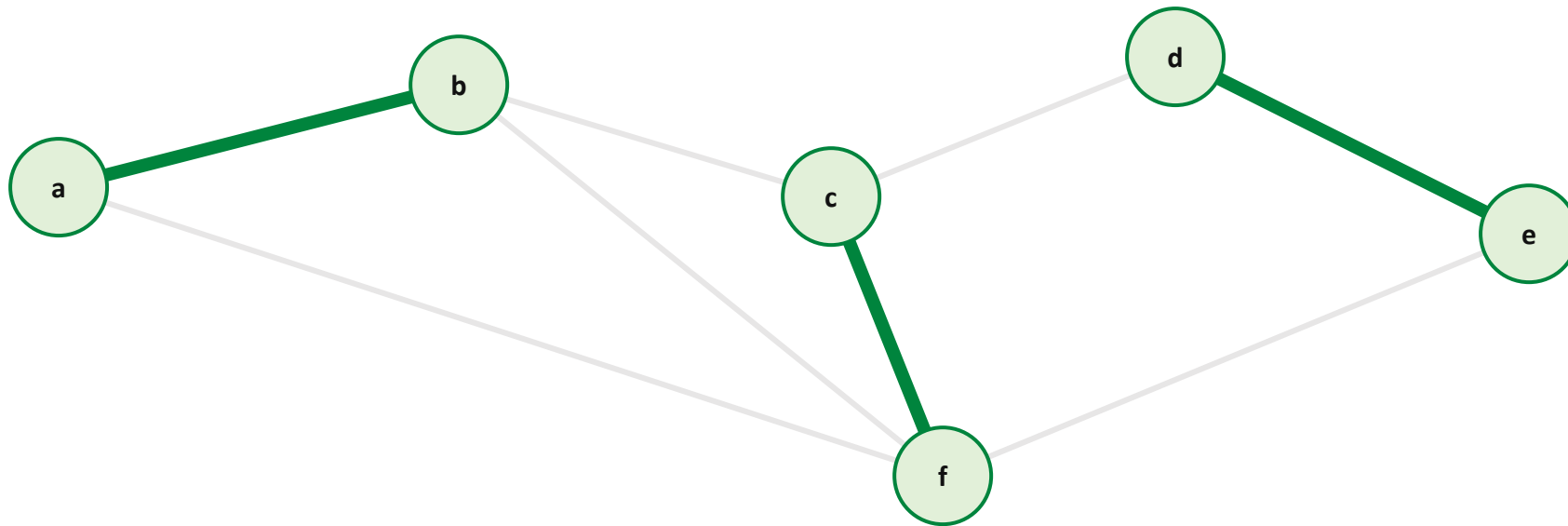
Delete edges incident to a or b; then choose $\{d,e\}$.



Matched edges never share an endpoint.

Build 3: add (c,f) to the matching

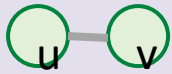
The edge (c,f) is disjoint from the first two matched edges, so add it to M.



Now M is maximal, and its endpoints form a vertex cover.

Why matched endpoints cover every edge

Suppose $\{u,v\}$ is uncovered



$u \notin C$

$v \notin C$

Neither endpoint is incident to a matched edge.

So $\{u,v\}$ is disjoint from M .



Then $\{u,v\}$ could be added to M

contradicts maximality

Therefore every edge has a selected endpoint.

How is OPT related to $|M|$ for any matching M ?

A matching lower-bounds every vertex cover

The matching edges are disjoint, so a cover needs a distinct chosen endpoint for each one.



one distinct cover vertex per matched edge

For every matching M : $|M| \leq \text{OPT}$.

Both endpoints give factor 2

maximal matching M

matching edges are disjoint

each edge forces one OPT vertex



$$|C| = 2|M|$$

$$\leq 2 \cdot \text{OPT}$$

2-approximation

Feasibility comes from maximality; the guarantee comes from disjointness.

Quick check: does maximum mean optimal?

Claim

Endpoints of a maximum matching always form an optimal cover.

TRUE/FALSE?

Quick check: does maximum mean optimal?

Claim

Endpoints of a maximum matching always form an optimal cover.

FALSE

What is true

Endpoints of any maximal matching cover all edges.

$\leq 2 \cdot \text{OPT}$

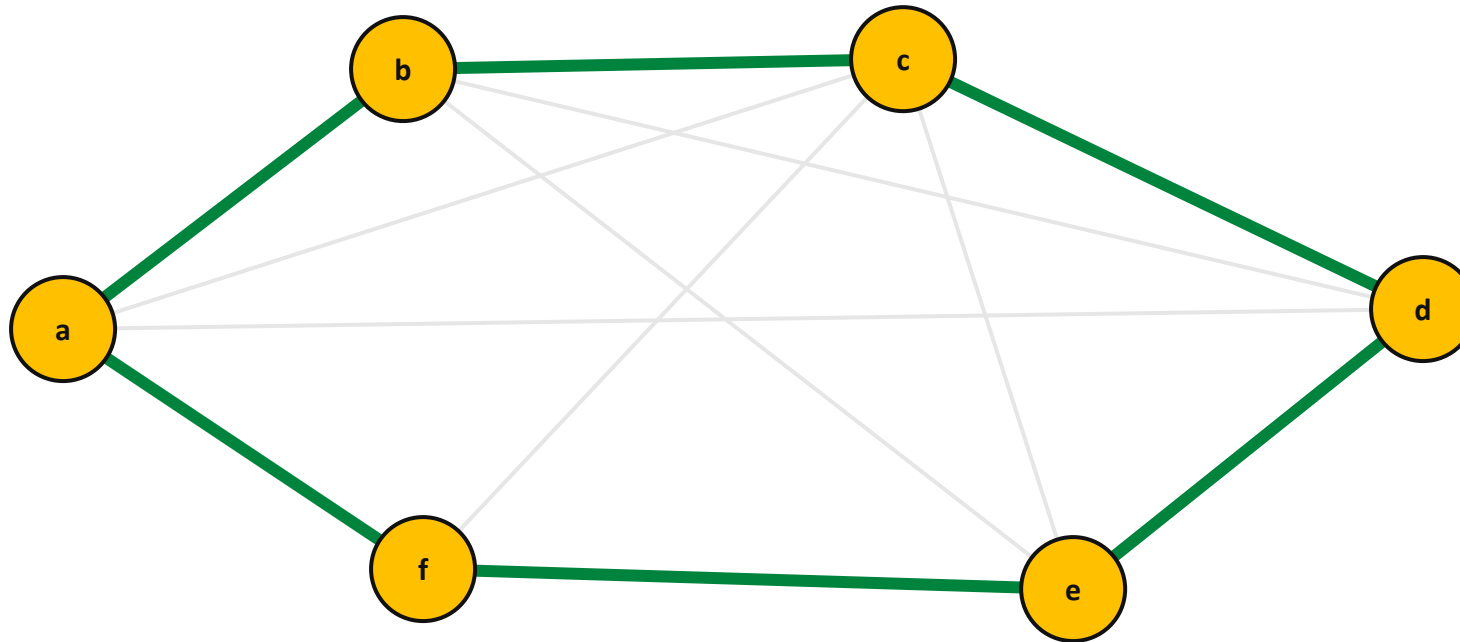
Maximum helps the lower bound; maximality proves feasibility.

Traveling Salesperson Problem (TSP)



TSP: visit every vertex once and return

Input: a complete weighted graph. Goal: a minimum-cost tour (Hamiltonian cycle).



A tour uses every vertex exactly once before returning to the start.

TSP: Brute Force and DP

- Brute force: Try $n!$ (or $(n - 1)!$ by fixing one node) permutations on nodes ($|V| = n$)
 - Run time $O(n!)$
- Try Dynamic Programming (Held-Karp): $O(n^2 2^n)$

Cost structure changes what is possible

General TSP

Arbitrary edge costs can create an enormous YES/NO gap.

no factor



Metric TSP

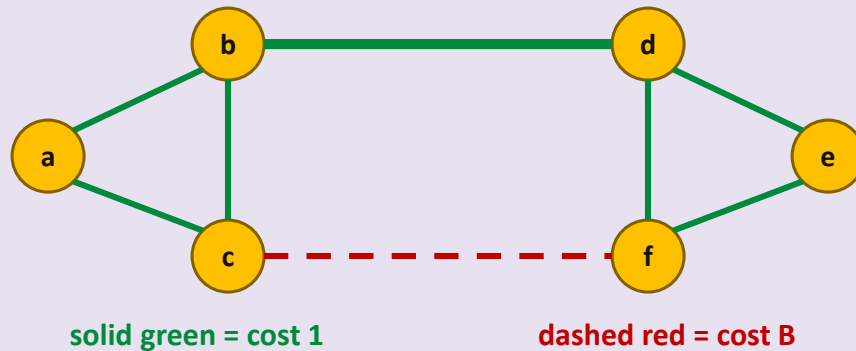
Triangle inequality makes shortcuts safe.

constant factor

Triangle inequality is what enables approximation.

Informal: A bridge creates an arbitrarily large TSP cost gap

NO instance: the cost-1 graph has a bridge



Complete the graph: cost 1 on edges of G ;
cost B on every non-edge.

YES: a Hamiltonian cycle has cost $n = 6$

**NO: a cycle must cross the cut twice;
only one cost-1 edge crosses it, so $\text{OPT} \geq B + 5$**

Choose $B = rn + 1$: an r -approximation would separate YES from NO (unless $P = NP$).

Metric TSP: the structural ingredients

Input

Complete graph with nonnegative symmetric costs.

Structure

$c(u,w) \leq c(u,v) + c(v,w)$ **triangle inequality**

Consequence

Shortcut repeated vertices without increasing cost.

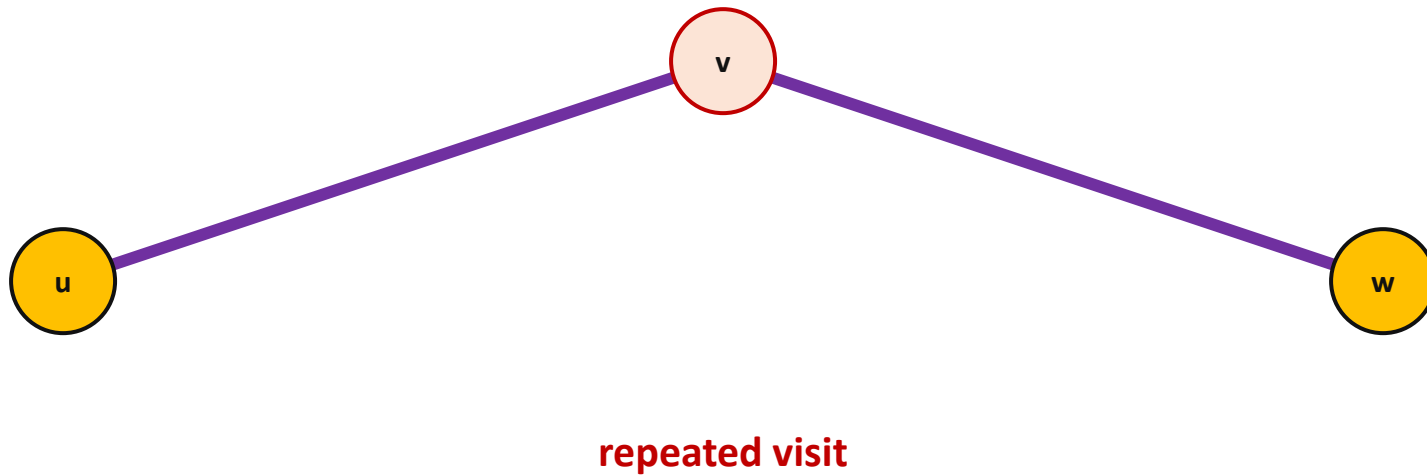
First build an easy repeated walk; then turn it into a tour.

Metric TSP is also NP-Hard

- Metric TSP is NP-hard
- **To think:** Where does the hardness come from?
 - Every vertex needs to be visited “**exactly**” once
 - What about “**at least**” once instead? (**TSP-with Repetition**)

A repeated walk contains a detour

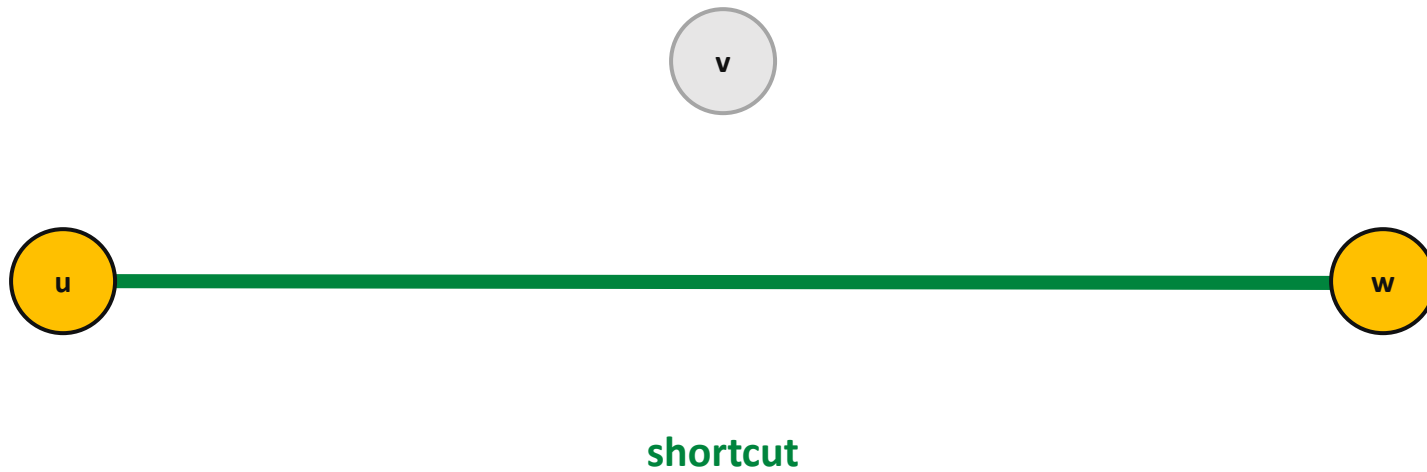
Suppose v has already been visited and the walk uses $u \rightarrow v \rightarrow w$.



Triangle inequality tells us the detour is never cheaper.

Shortcut the repeated vertex

Replace $u \rightarrow v \rightarrow w$ by the direct edge $u \rightarrow w$.

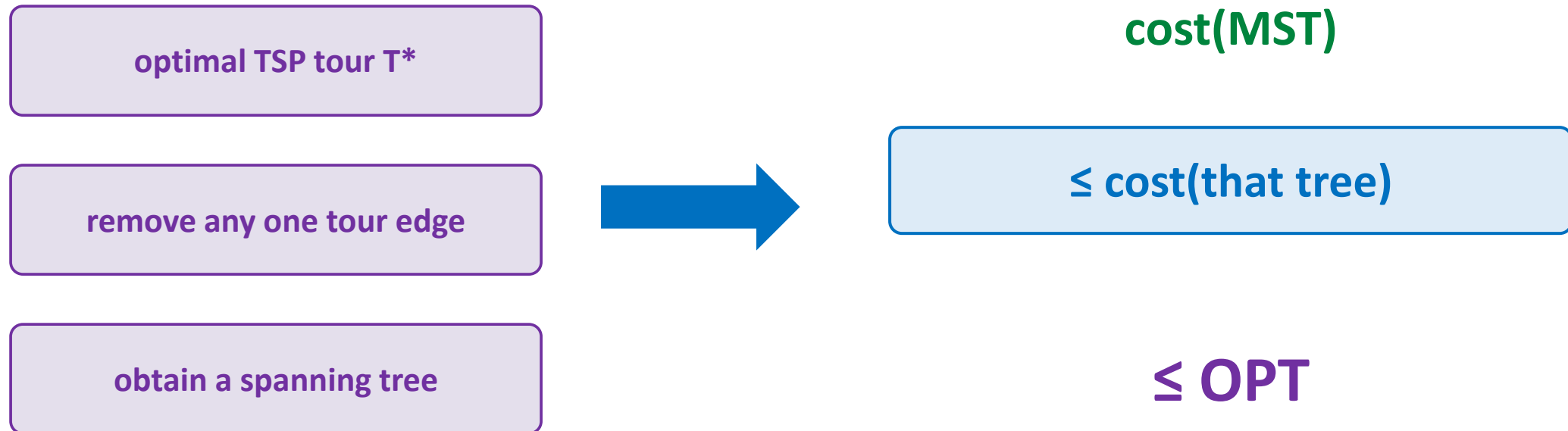


$$c(u,w) \leq c(u,v) + c(v,w)$$

Can we solve Metric TSP with Repetition?

- Again, NP-hard
- But can we get an approximation?
- Think about some “object” that connects all the vertices “with minimum cost”
- Have you encountered any such object before?

An MST (minimum spanning tree) is a lower bound on OPT



This lower bound will pay for the spanning part of both TSP algorithms.

Double-tree: MST $\rightarrow$ Euler walk $\rightarrow$ tour

H = minimum spanning tree



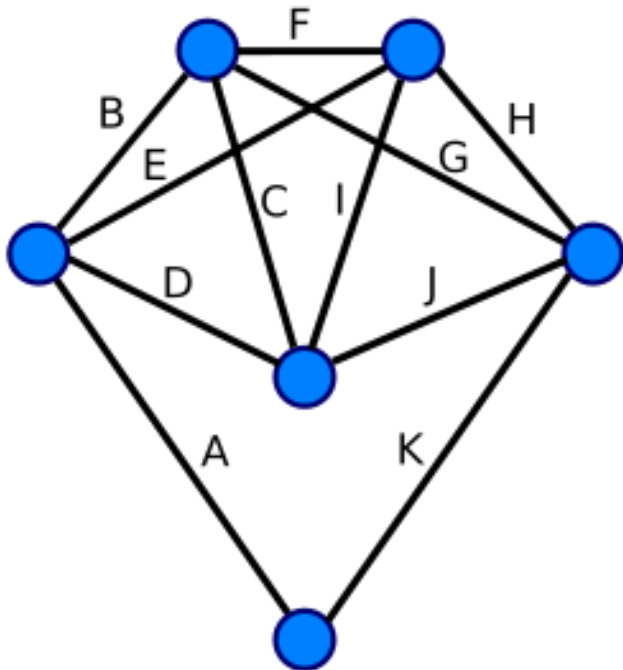
Output a Hamiltonian tour

- 1 Duplicate every edge of H
- 2 Find an Eulerian cycle
- 3 Shortcut repeated vertices

The next four slides build the algorithm one layer at a time.

Eulerian Cycle

- **Definition:** In a multigraph $G = (V, E)$, an **Eulerian Cycle** C is a cycle (sequence of edges) such that
 - Every edge $(u, v) \in E$ appears in C exactly the same number of times as in E (basically, each occurrence of edge $(u, v) \in E$ appears exactly once)



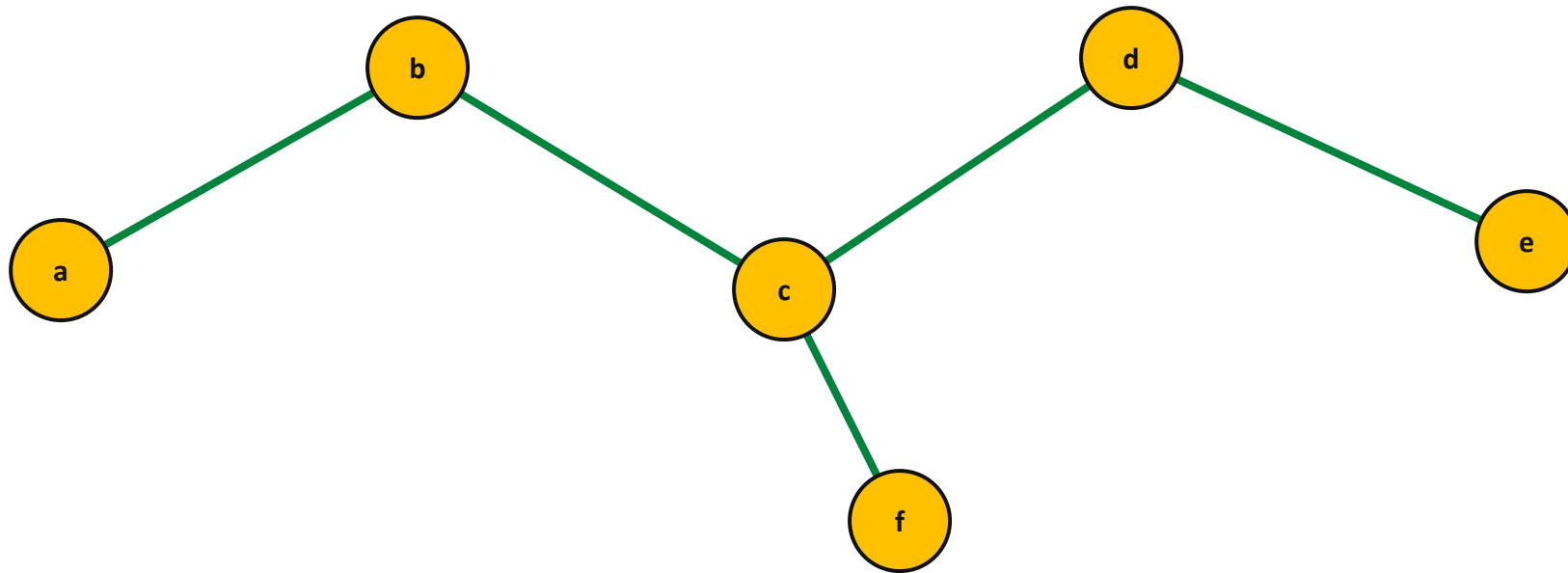
Eulerian Cycle: A, B, C, D, E, F, G, H, I, J, K

Eulerian Cycle: Existence and Algorithm

- **Theorem:** A multigraph $G = (V, E)$ has an Eulerian cycle if and only if
 - It is connected (except isolated vertices)
 - Each vertex has even degree
- Moreover, there is a polynomial time algorithm that, given $G = (V, E)$ finds an Eulerian cycle if exists.
- **Proof:** Exercise (Try using induction on the number of edges)

Build 1: compute a minimum spanning tree

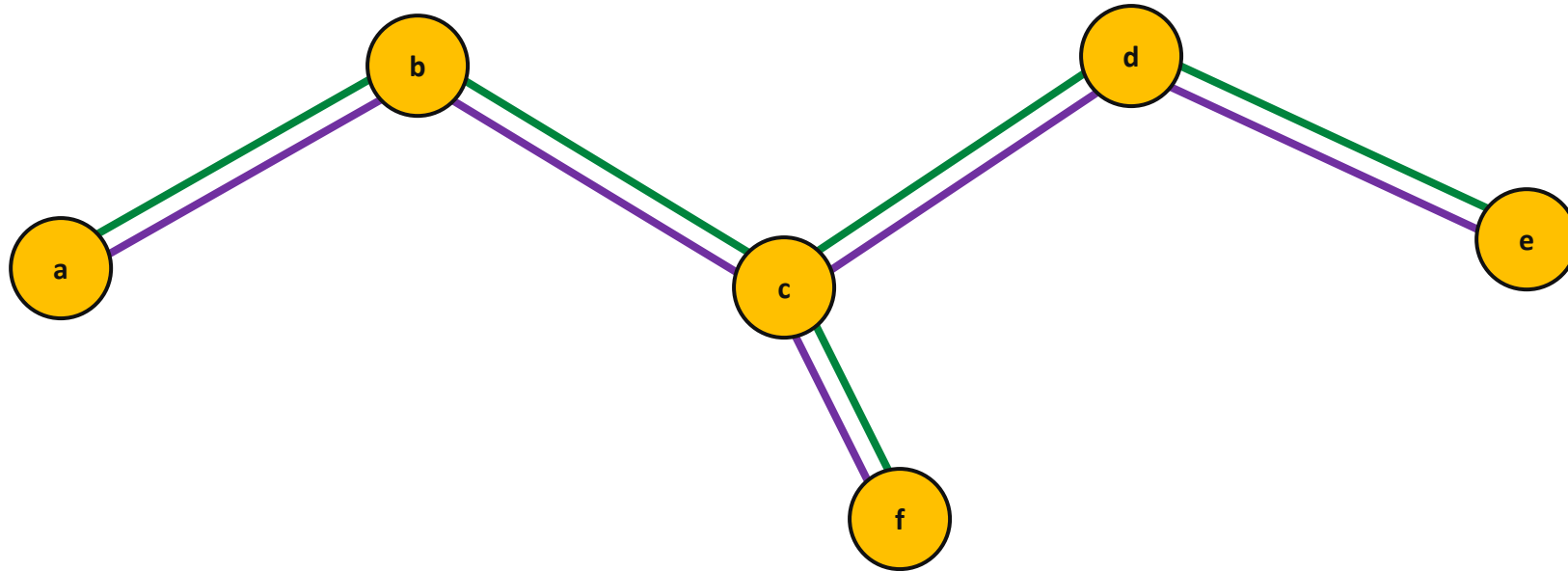
The MST connects every vertex at cost at most OPT.



Spanning lower bound: $\text{cost}(H) \leq \text{OPT}$.

Build 2: duplicate every tree edge

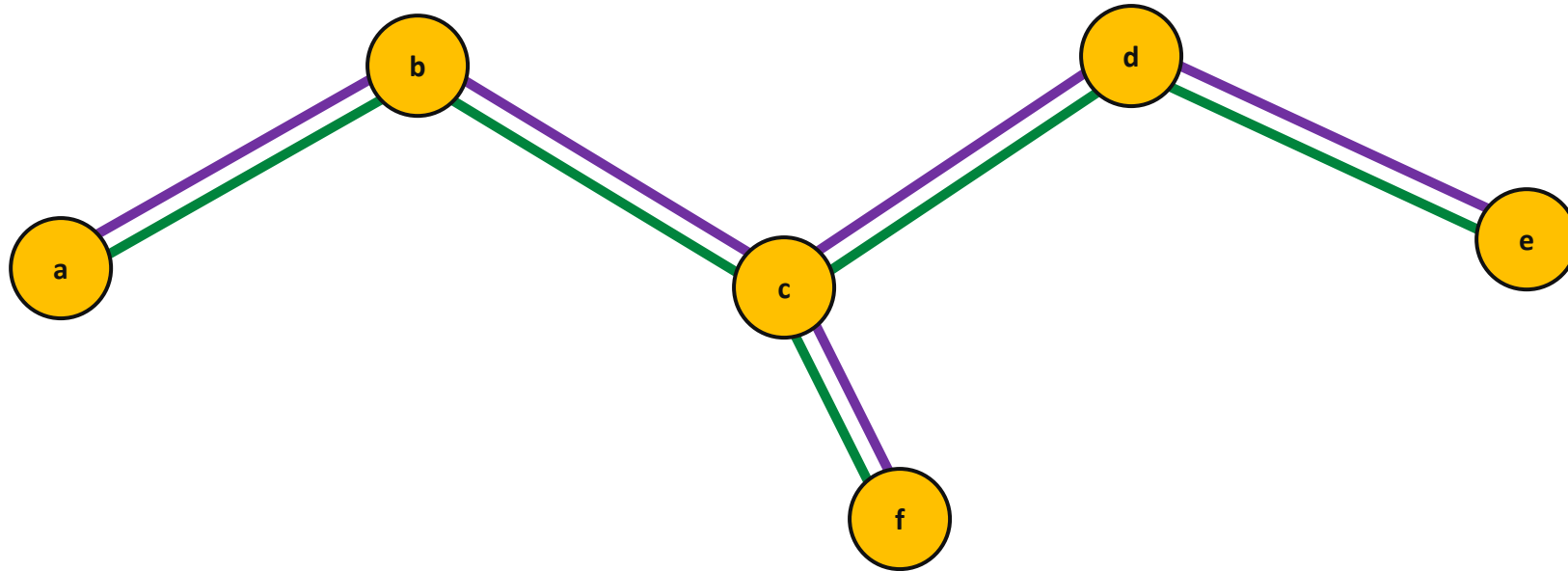
Every degree becomes even; total multigraph cost is $2 \cdot \text{cost}(H)$.



Parity is now fixed—but every tree edge was bought twice.

Build 3: traverse an Eulerian cycle

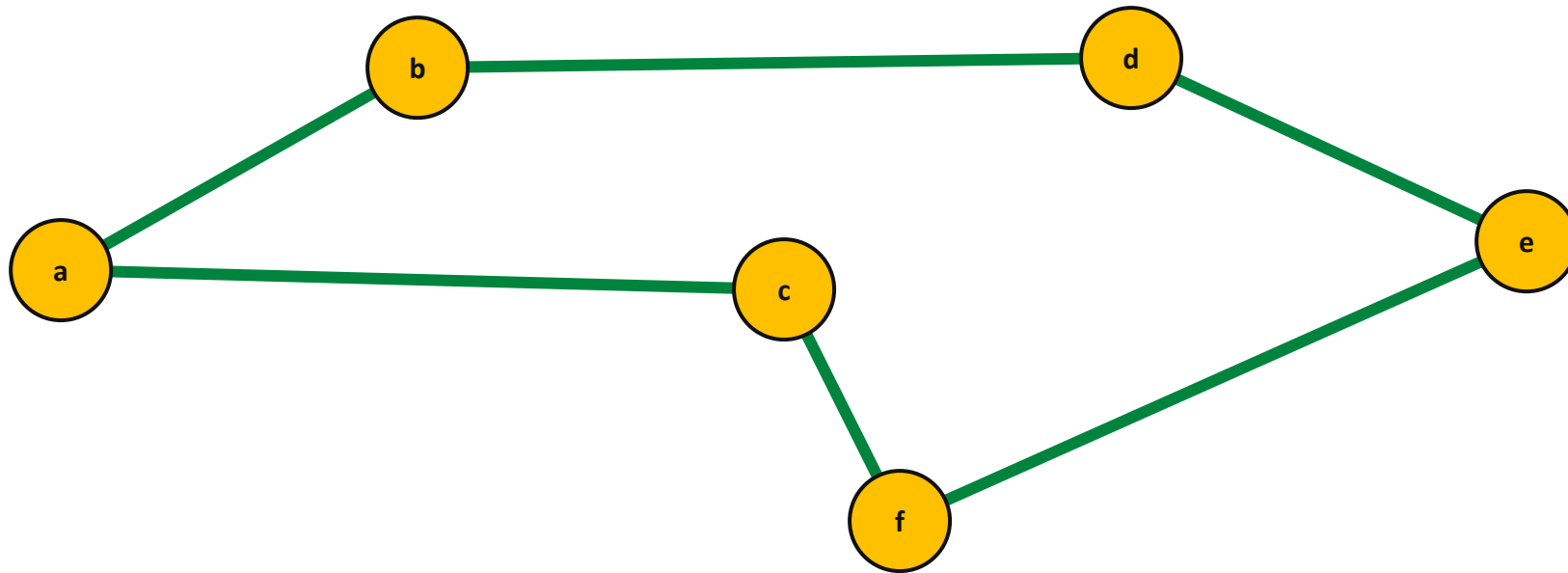
A connected even-degree multigraph has an Eulerian cycle.



The walk uses every copied edge exactly once.

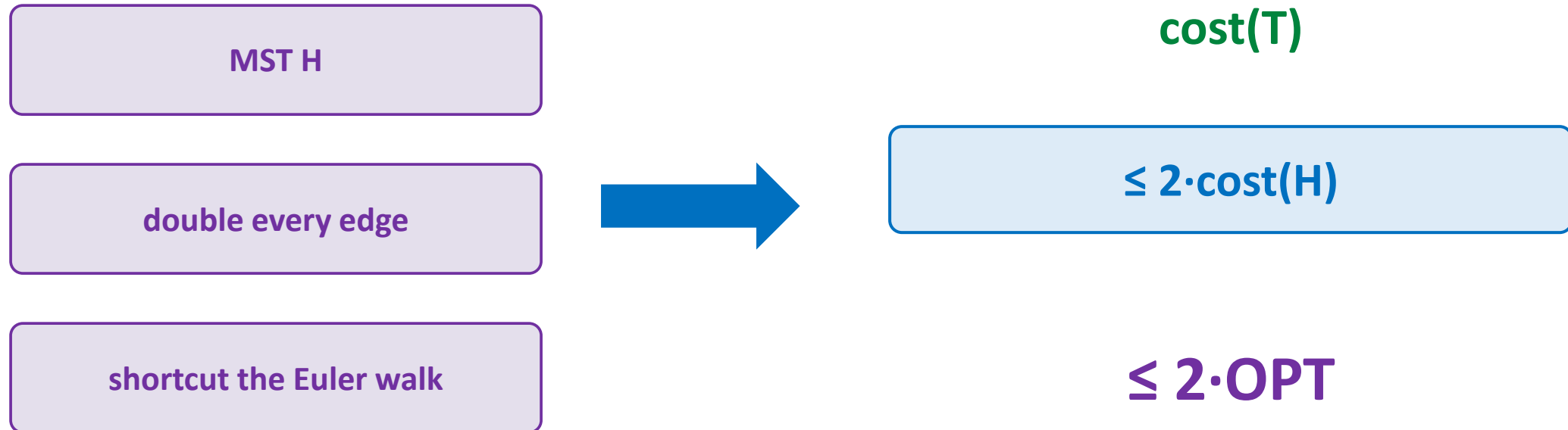
Build 4: shortcut repeated vertices

Remove repeated visits using triangle inequality.



The shortcut tour costs no more than the Euler walk.

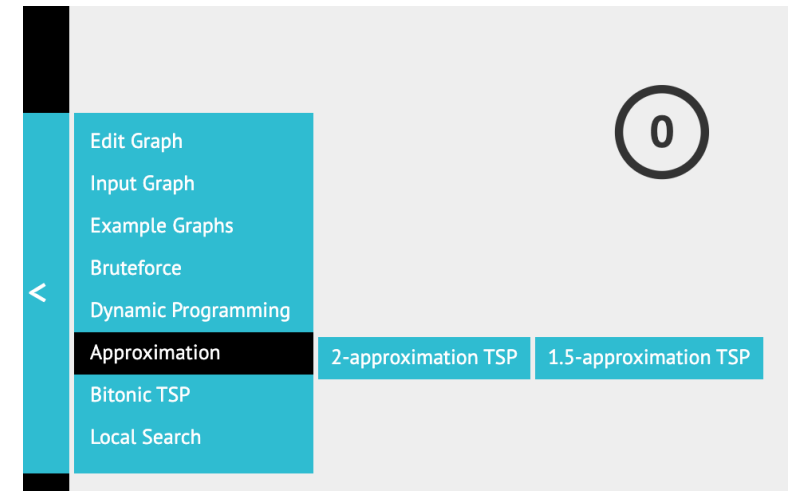
Double-tree costs at most $2 \cdot \text{OPT}$



The two losses are visible: double the tree, then never increase cost again.

Exercise

- Try <https://visualgo.net/en/tsp>



- Design an example such that the above algorithm cannot get “asymptotically” better than 2-factor approximation

Where does factor 2 come from?

Double-tree

Buy a second copy of every MST edge.

+ $\text{cost}(H)$



Observation

Only odd-degree vertices block an Eulerian cycle.

repair parity

Pay only for the vertices whose parity is wrong.

Eulerian cycles are controlled by parity

Criterion

Connected multigraph is Eulerian iff every degree is even.

Double-tree

Duplicating all edges makes every degree even.

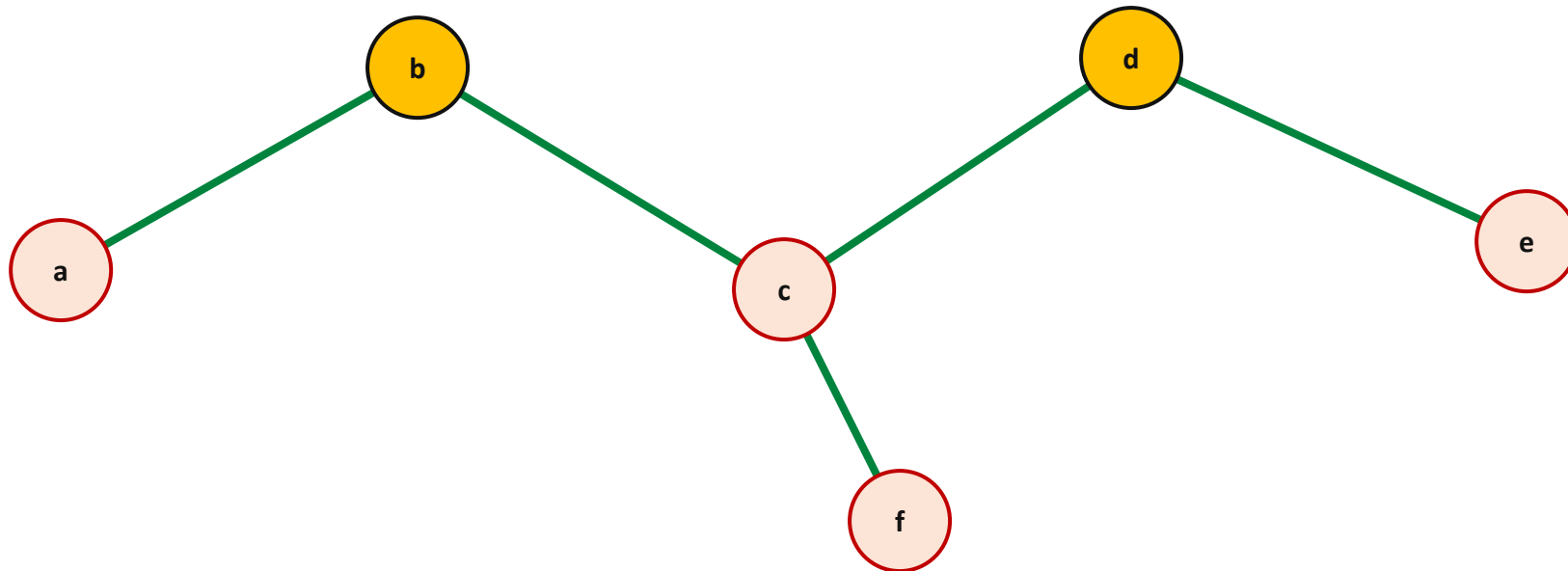
Christofides

Add edges only between odd-degree vertices.

Augmentation is a parity-repair problem.

Only odd-degree vertices need repair

In this MST, the red vertices have odd degree.



Adding one edge flips parity at both endpoints.

Why the number of odd vertices is even

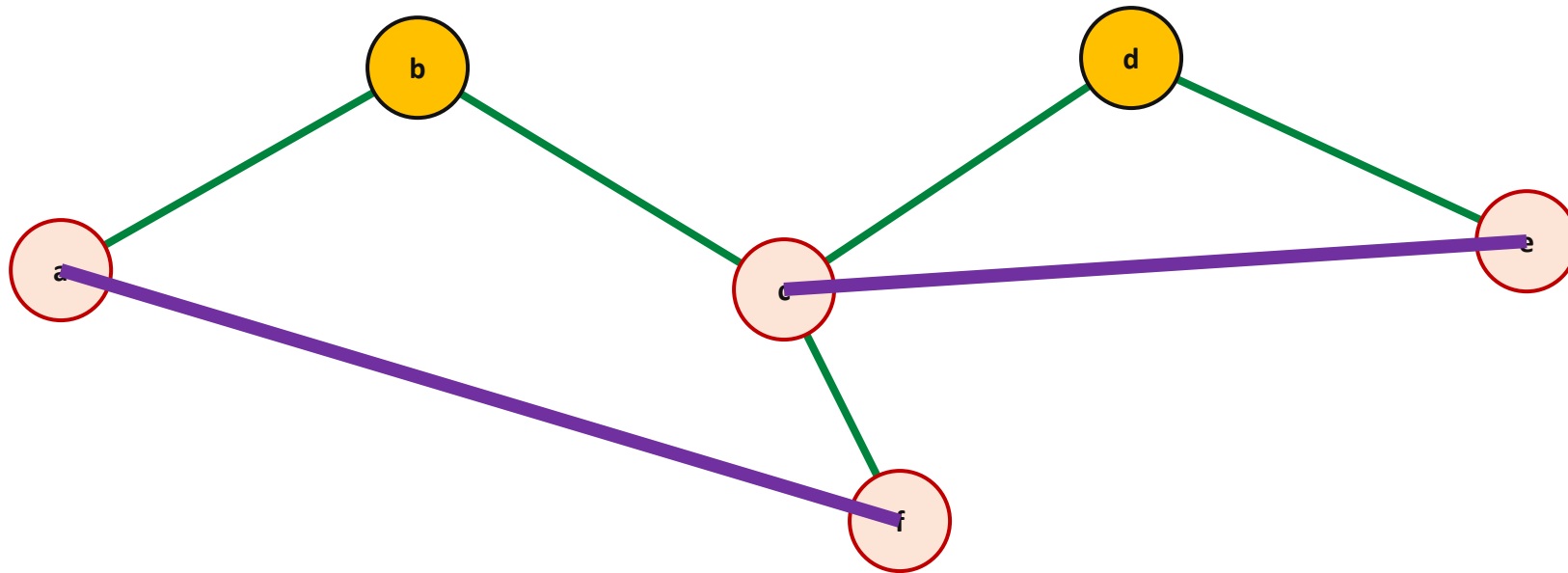
$$\sum_{v \in V} \text{degree}(v) = 2|E| \text{ (always even)}$$

therefore #odd is even

Odd vertices can always be paired by a perfect matching.

A perfect matching repairs all odd degrees

Pair every odd-degree vertex exactly once; choose the minimum-cost pairing.



After adding M , every degree in $H \cup M$ is even.

Christofides: repair parity cheaply

$H = \text{MST}$



$O = \text{odd-degree vertices of } H$

- 1 Find a minimum perfect matching M on O
- 2 **Find an Euler tour of $H \cup M$**
- 3 Shortcut repeated vertices

The matching replaces the second copy of the whole tree.

Christofides spends in two phases

Spanning

Minimum spanning tree H
connects all vertices.

$\leq \text{OPT}$



Parity repair

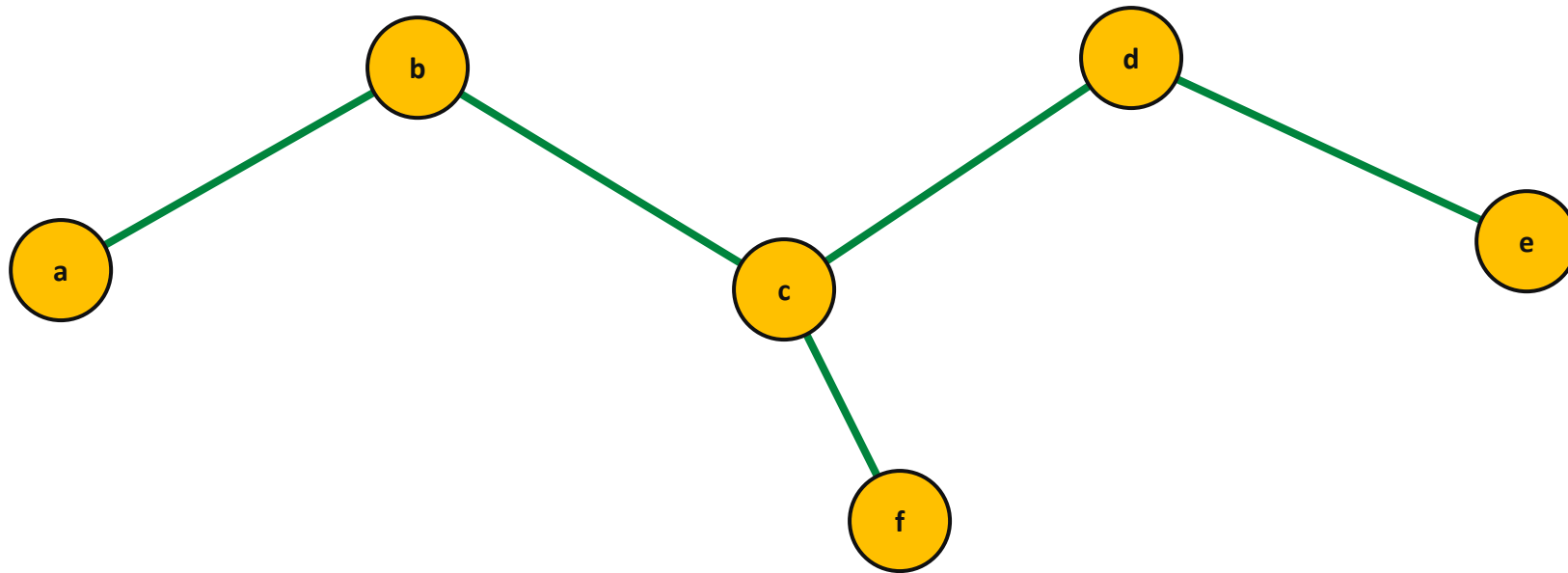
Minimum perfect matching M fixes
the odd set O .

$\leq \text{OPT}/2$

Then Euler tour + shortcut converts $H \cup M$ into a TSP tour.

Build 1: compute the MST H

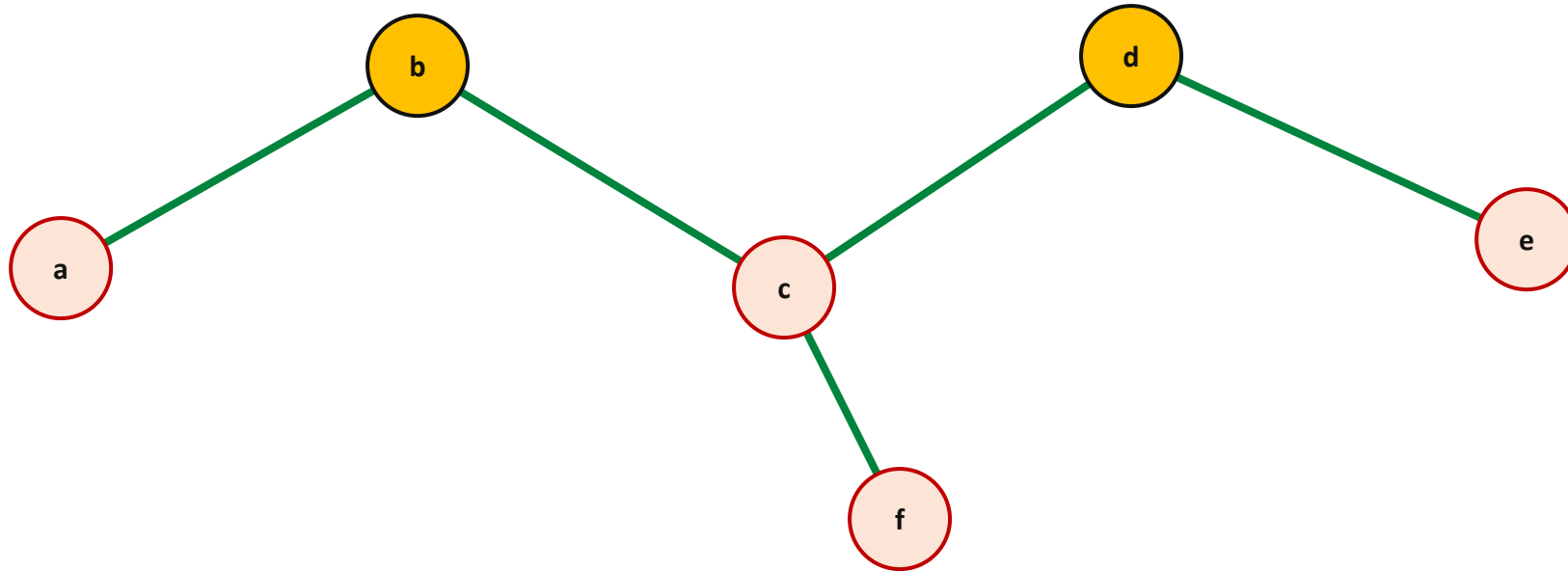
The spanning cost satisfies $\text{cost}(H) \leq \text{OPT}$.



Spanning budget: at most OPT.

Build 2: mark the odd-degree set O

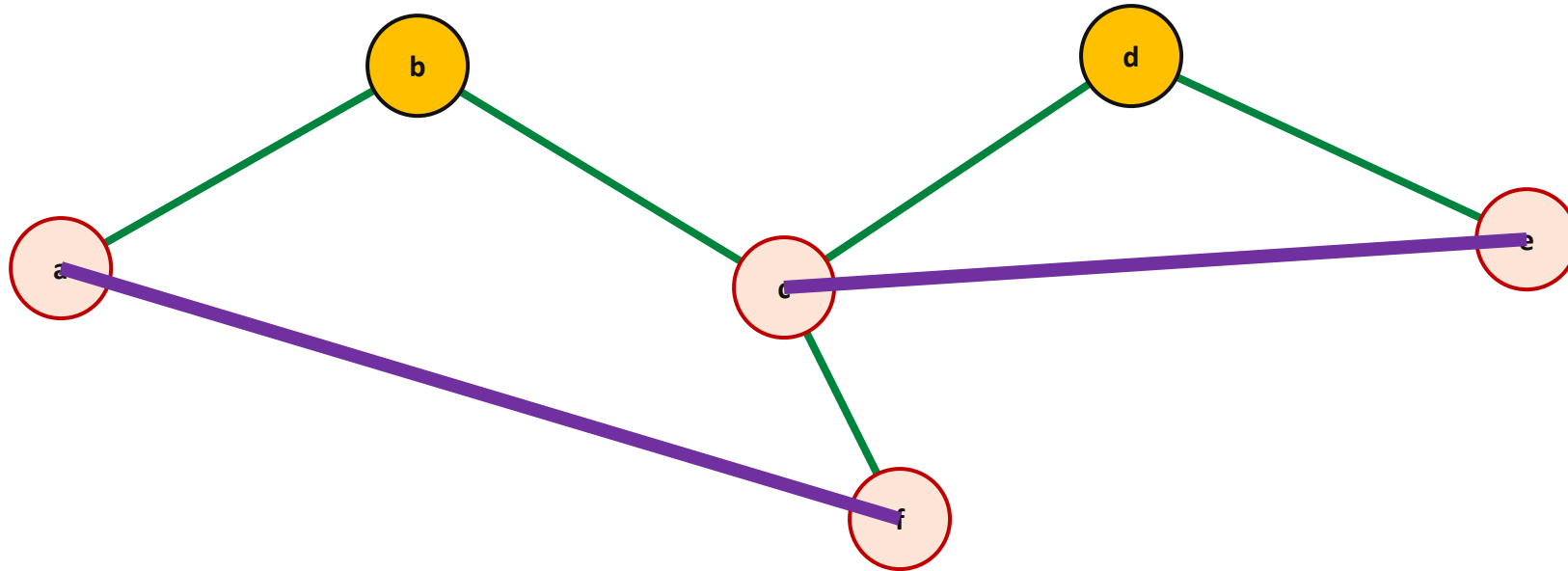
Only vertices in O need their parity flipped.



The handshaking lemma guarantees $|O|$ is even.

Build 3: add a minimum matching on O

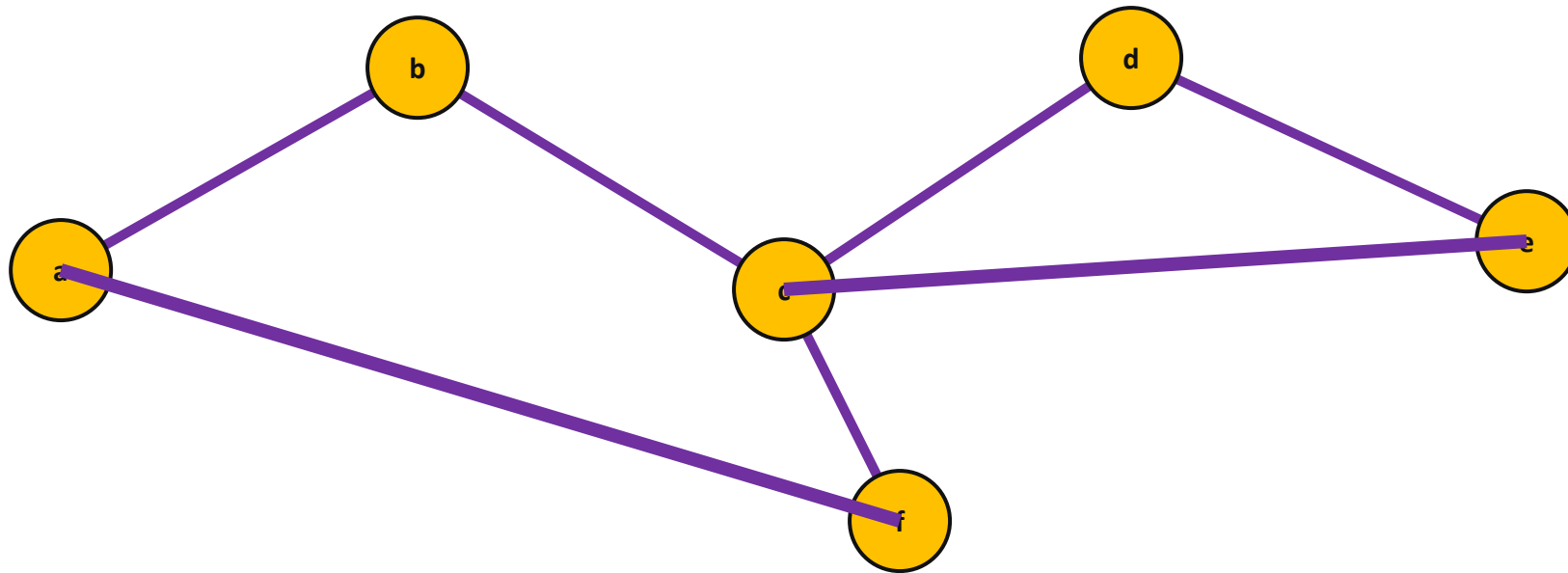
Each matching edge flips two odd degrees to even.



Repair budget: prove $\text{cost}(M) \leq \text{OPT}/2$.

Build 4: traverse H U M with an Euler tour

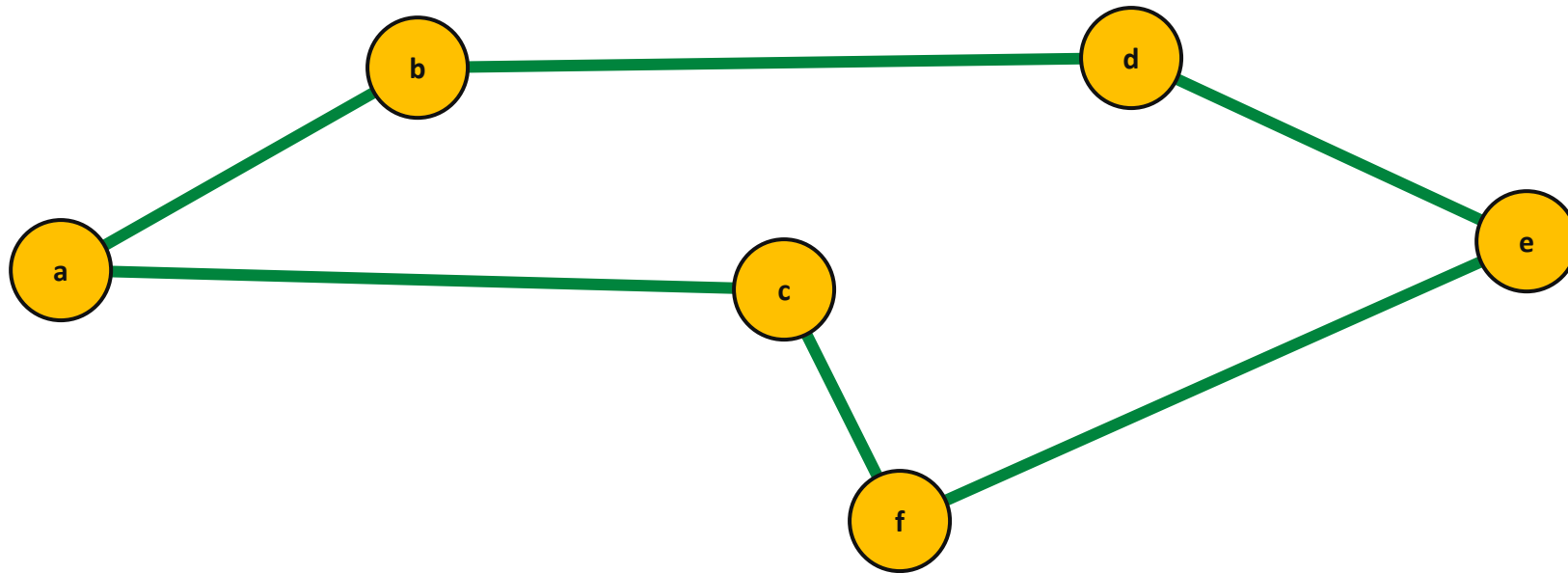
Every degree is now even, so an Eulerian cycle exists.



The Euler walk costs $\text{cost}(H) + \text{cost}(M)$.

Build 5: shortcut to a Hamiltonian tour

Triangle inequality removes repeated visits for free.



Final tour cost never exceeds $\text{cost}(H) + \text{cost}(M)$.

The 3/2 proof has two budgets

Spanning

$$\text{cost}(H) \leq \text{OPT}$$

Parity

$$\text{cost}(M) \leq \text{OPT}/2$$

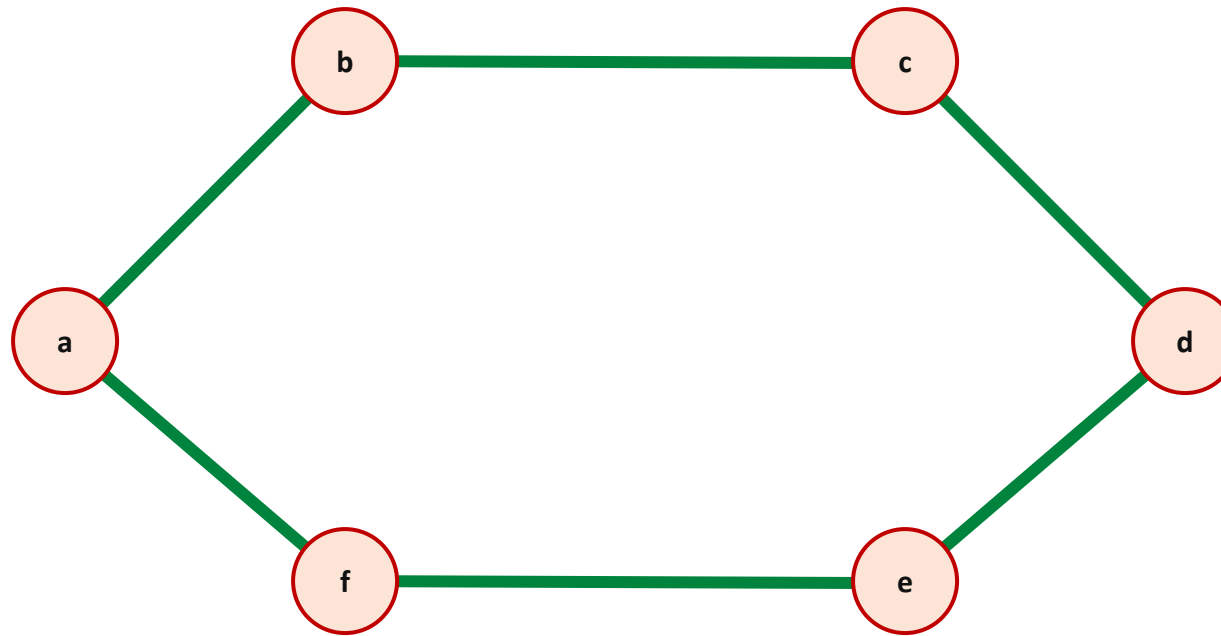
Shortcut

$$\text{cost}(T) \leq \text{cost}(H) + \text{cost}(M)$$

It remains to justify the matching budget.

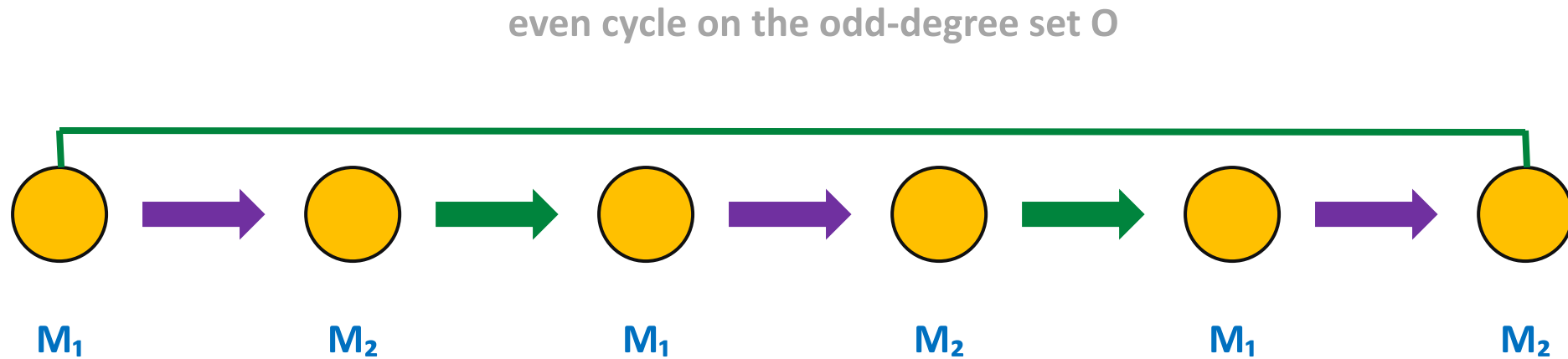
Shortcut OPT down to the odd-degree set

Keep only the vertices in O and shortcut over every other vertex of an optimal tour T^* .



The resulting even cycle T^* has cost at most OPT .

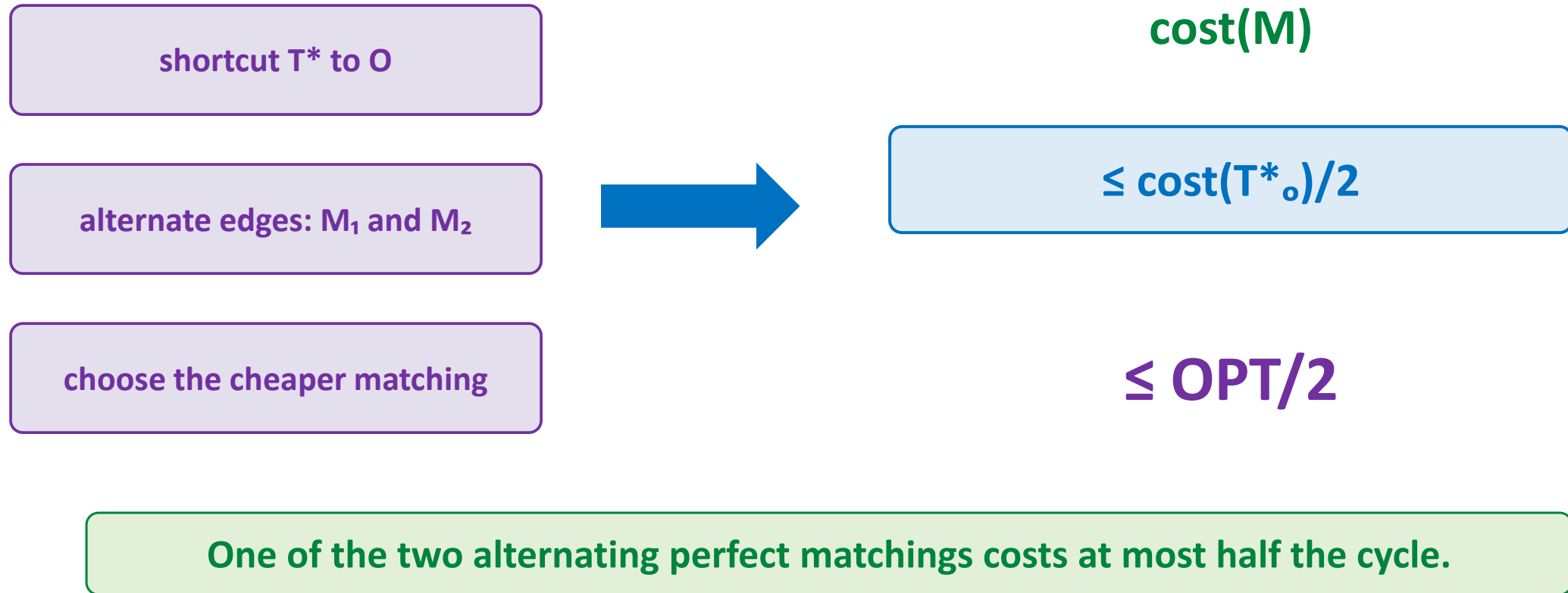
Alternating edges form two perfect matchings



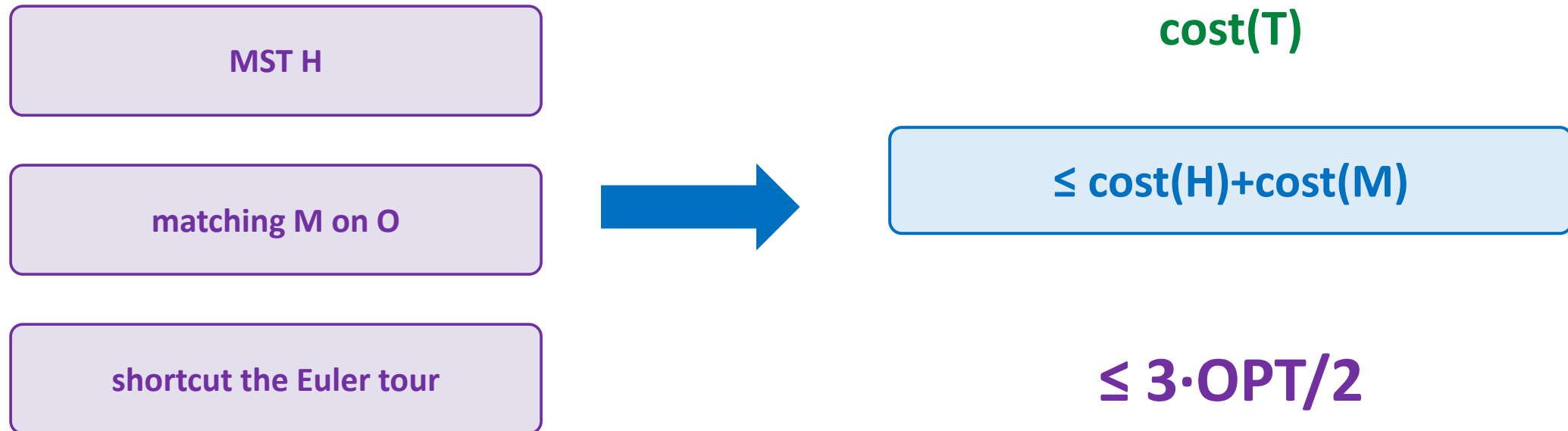
Purple and green edges each touch every vertex exactly once.

The cheaper alternating matching costs at most half the cycle.

The matching costs at most $\text{OPT}/2$



Christofides Algorithm: costs at most $3 \cdot \text{OPT}/2$



The spanning and parity budgets add; shortcutting never hurts.

Compare the two metric-TSP algorithms

Double-tree

MST + a second copy of every tree edge.

2-approx



Christofides

MST + minimum matching on odd vertices.

3/2-approx

Both finish with an Euler tour and safe shortcutting.

Exercise

- Design an example such that the above algorithm cannot get “asymptotically” better than $3/2$ -factor approximation
- **Remark:** Can get better than $3/2$: “A (slightly) improved approximation algorithm for metric TSP” by [Karlin, Klein, Gharan’21](#)

One proof skeleton, three guarantees

Vertex Cover

maximal matching or LP lower bound $\rightarrow$ factor 2

Double-tree

MST lower bound + full parity repair $\rightarrow$ factor 2

Christofides

MST + minimum matching repair $\rightarrow$ factor $3/2$

Design the solution around a lower bound that can pay for every step.

References (so far)

- **Knapsack:** Chapter 8 of [Vazirani]
- **Set Cover:** Chapter 2 of [Vazirani]
- **Traveling Salesperson Problem:** Chapter 3 of [Vazirani]